UNIVERSITY OF SOUTHAMPTON
Faculty of Engineering and Physical Sciences
School of Electronics and Computer Science

A project report submitted for the award of
BEng Electronic Engineering

Supervisor: Dr Tomasz Kazmierski
Examiner: Professor Nicholas Harris

Design and Implementation of a RISC-V Sparse Matrix Accelerator on FPGA

by Tochi C. Anyanwu

April 21, 2026

UNIVERSITY OF SOUTHAMPTON

ABSTRACT

FACULTY OF ENGINEERING AND PHYSICAL SCIENCES
SCHOOL OF ELECTRONICS AND COMPUTER SCIENCE

A project report submitted for the award of BEng Electronic Engineering

by Tochi C. Anyanwu

Sparse matrix–dense matrix multiplication (SpMM) is a key kernel in graph neural networks, recommender systems, and compressed neural network inference, but it performs poorly on general-purpose processors because of irregular memory access and low arithmetic intensity. This report presents the design, implementation, and evaluation of a PicoRV32 RISC-V system-on-chip with a custom hardware accelerator for compressed sparse row (CSR) SpMM on the Terasic DE1-SoC Cyclone V FPGA. The design was developed in three optimisation stages: a baseline accelerator with tile length ($TN = 8$) parallel MAC lanes across the dense output dimension, a DSP-oriented MAC enhancement that maps each lane to a dedicated Cyclone V multiplier-accumulator block, and a runtime symmetric INT8 packing stage that stores four quantised operands per 32-bit RAM word. A 12-case hardware benchmark suite evaluating matrix size, dense width, sparsity level, and nonzero pattern was executed in firmware on the synthesised SoC, and all cases passed element-by-element correctness checks. The final INT8 design achieved a peak speedup of 56.80× over the quantised PicoRV32 softcore and 11.92× over an ARM Cortex-M0 softcore with a 2.87× cycle reduction over baseline.

Statement of Originality

- I have read and understood the ECS Academic Integrity information and the University's Academic Integrity Guidance for Students.
- I am aware that failure to act in accordance with the Regulations Governing Academic Integrity may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

You must change the statements in the boxes if you do not agree with them.

We expect you to acknowledge all sources of information (e.g. ideas, algorithms, data) using citations. You must also put quotation marks around any sections of text that you have copied without paraphrasing. If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

I have acknowledged all sources, and identified any content taken from elsewhere.

If you have used any code (e.g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I have used the open-source picoRV32 RISC-V core by YosysHQ as the soft core processor within my system on chip design.
I has also used the Arm cortex M0 from Arm LTD.

You can consult with module teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly-accessible repositories e.g. Github, unless expressly permitted by the module leader), or help them to do theirs. For individual assignments, we expect you to work on your own. For group assignments, we expect that you work only with your allocated group. You must get permission in writing from the module teaching staff before you seek outside assistance, e.g. a proofreading service, and declare it here.

I did all the work myself, or with my allocated group, and have not helped anyone else.

We expect that you have not fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (either in the report, or submitted as a separate file) so that your results could be reproduced.

The material in the report is genuine, and I have included all my data/code/designs.

We expect that you have not previously submitted any part of this work for another assessment. You must get permission in writing from the module teaching staff before re-using any of your previously submitted work for this assessment.

I have not submitted any part of this work for another assessment.

If your work involved research/studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report, and list the ethical approval reference number(s) in the box below.

My work did not involve human participants, their cells or data, or animals.

ECS Statement of Originality Template, updated August 2018, Alex Weddell aiofficer@ecs.soton.ac.uk

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr Tomasz Kazmierski, for his continuous guidance, insightful technical feedback, and encouragement throughout this project. His direction shaped the design approach and helped maintain the clarity and rigour of the evaluation methodology. I am also deeply thankful to my family and friends for their constant support and patience, particularly during the intensive integration and debugging phases of the FPGA deployment.

Contents

Chapter 1

Introduction

1.1 Motivation

Sparse matrix–dense matrix multiplication (SpMM) is one of the fundamental computational kernels in modern machine learning and graph-based computing. In graph neural networks (GNNs) it embodies the neighbourhood aggregation step, in which a sparse adjacency matrix is multiplied against a dense feature matrix [1, 2]. In large-scale recommender systems, collaborative filtering relies on sparse user–item interaction matrices whose dimensions can reach hundreds of millions of rows [3]. In scientific simulation it is the inner loop of iterative solvers for finite-element and finite-difference discretisations. The same pattern appears in transformer attention with sparse masks and in the weight-matrix multiply step of pruned and compressed neural networks [4]. Across all of these workloads the common denominator is that the useful arithmetic is only a small fraction of the work that a general-purpose processor actually performs, with most cycles spent chasing indirect indices, handling row boundaries, and issuing scattered memory accesses.

With the rapid rise in demand for AI, there has been a matching rise in demand for better model algorithms, more data, and above all more compute. On the compute side, most AI workloads are now executed on application specific integrated circuits (ASICs), since these have been shown to maximise throughput, energy efficiency, and cost effectiveness for the particular matrix operations that AI models need to run. Some of the most well known commercial examples are the Tensor Processing Unit (TPU) from Google and the Neural Processing Unit (NPU) from Apple. The design of any such ASIC, however, almost always begins with prototyping on

a Field Programmable Gate Array (FPGA), because FPGAs can execute many operations in parallel and are easily reconfigurable, which keeps the cost and turnaround time of architectural exploration both low and fast. An FPGA-hosted accelerator for SpMM is therefore a natural first step towards a fully custom silicon implementation, and is the platform targeted by this project.

1.2 Project Aim

The aim of this project is to design, implement, and evaluate a RISC-V compatible hardware accelerator for compressed sparse row (CSR) SpMM on the Terasic DE1-SoC FPGA board. The accelerator must integrate cleanly into a small system-on-chip (SoC), share on-chip block RAM with the CPU, and deliver a measurable, reproducible speedup over a comparable software implementation while remaining understandable at the register-transfer level (RTL) and transparent in its benchmarking methodology. A secondary aim is to isolate the individual contribution of each optimisation step, so that the dominant performance bottleneck can be identified through direct measurement rather than assumption. To achieve this, the design is developed in three stages (a baseline accelerator, a parallel DSP-oriented multiply-accumulate enhancement, and a symmetric INT8 packing stage), forming a controlled experimental path in which only one design dimension is changed at a time.

Beyond the core aim, the project defined three stretch goals at the outset. The first was to introduce a custom RISC-V instruction to trigger the accelerator directly from the CPU pipeline, rather than through a memory-mapped register write, which would reduce launch overhead and demonstrate that the PicoRV32 softcore can be extended at the instruction set level. The second was to implement runtime INT8 quantisation so that operand fetch bandwidth could be reduced by packing four quantised values into a single 32-bit RAM word. The third was to extend the accelerator with optional activation functions and a max-pooling stage, so that it could serve as a building block for a small inference pipeline rather than as a standalone kernel. Of the three, the INT8 quantisation goal was achieved and is reported in full in this work; the remaining two are discussed as future work in Chapter 9.

1.3 Report Structure

The remainder of this report is structured as follows. Chapter **??** reviews the background needed to justify the design choices, covering SpMM workloads, sparse data formats, prior accelerator work, the rationale for FPGAs, and INT8 quantisation theory. Chapter 3 defines the implemented SoC platform and its memory-mapped I/O interface. Chapter 4 presents the accelerator architecture and each optimisation step in detail. Chapter 5 describes the bare-metal firmware and data preparation flow. Chapter 6 defines the verification and benchmark methodology. Chapter 7 presents and interprets all recorded benchmark results. Chapter 8 discusses lessons learned and project management, and Chapter 9 closes the report with a summary and proposed future work.

Chapter 2

Background and Literature Review

There is a vast body of algorithms for Sparse Matrix Multiplication (SpMM), as well as many different storage formats for converting sparse matrices into representations that can be operated on efficiently. This chapter provides the information needed to understand the context and design decisions taken throughout this project, alongside the goals that were set out at the start. If any of the material that follows is unfamiliar, a more elementary overview of SpMM and the CSR format is given in Appendix A, and further reading can be pursued through the references cited in each section.

2.1 SpMM in Modern Workloads

The motivation for a dedicated SpMM accelerator is made concrete by the sparsity levels that real workloads exhibit. Figure 2.1 summarises representative density ranges across four workload classes: in each case, the fraction of nonzero entries is small enough that storing and operating on the matrix in dense form would be wasteful by one to five orders of magnitude.

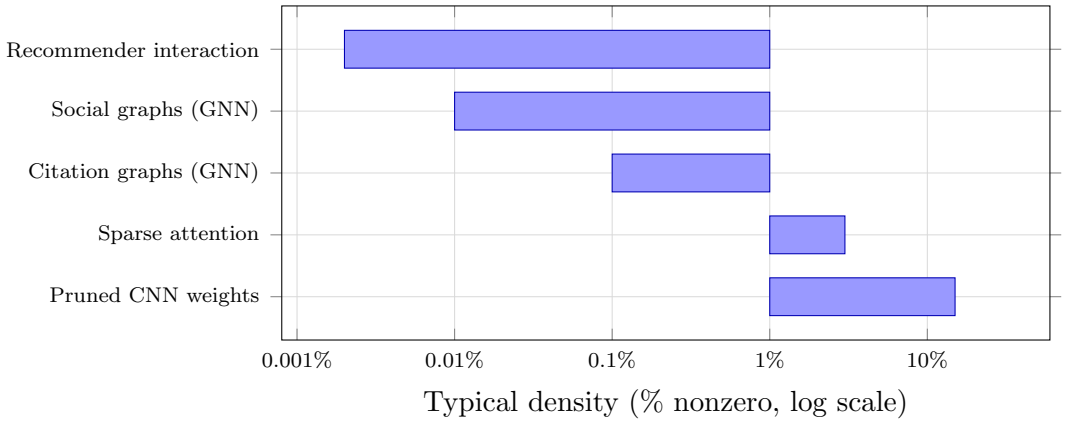

FIGURE 2.1: Representative density ranges for matrices encountered in four classes of modern workload. Social and citation graphs in GNN benchmarks such as `ogbn-arxiv` and `Reddit` typically have densities below 0.1% [1]; large-scale recommender interaction matrices are often two orders of magnitude sparser still [3]. Pruned CNNs sit at the other end of the range, retaining 10–20% of their weights after aggressive pruning [4]. In every case, an SpMM execution model that iterates only over the nonzeros avoids between 90% and $> 99.99\%$ of the arithmetic that a dense multiply would perform.

2.1.1 Graph Neural Networks

Graph neural networks (GNNs) are an important and growing class of workloads in which SpMM appears as a first-class primitive. Each layer of a message-passing GNN such as GraphSAGE [1] or a Graph Attention Network [2] computes

$$H^{(l+1)} = \sigma\left(\hat{A}\, H^{(l)}\, W^{(l)}\right),$$

where $\hat{A}$ is the normalised sparse adjacency matrix of the input graph and $H^{(l)}$ is the dense feature matrix. The product $\hat{A}\, H^{(l)}$ is a sparse–dense multiply that dominates inference time on large graphs.

2.1.2 Recommender Systems and Sparse Embedding Lookup

Industrial recommender systems such as the YouTube recommendation engine [3] maintain embedding tables with hundreds of millions of rows. The forward pass reduces to a row-gather SpMM on the interaction matrix, where sparse interaction patterns meet dense embedding rows.

2.1.3 Scientific Computing

In finite-element and finite-difference simulation, sparse matrices arise from the discretisation of differential operators on computational meshes. SpMM appears in multivector iterative solvers such as block GMRES and Lanczos methods, where each iteration applies the sparse system matrix to a block of dense vectors.

2.1.4 Compressed and Pruned Neural Networks

Weight pruning removes less-significant connections from a dense neural network, turning weight matrices into sparse ones. Han et al. [4] showed that networks pruned to 80–90% sparsity can be executed in compressed form with large energy and storage savings; combined with INT8 quantisation, as in the EIE design, SpMM on the compressed weights becomes the dominant kernel. The same principle applies to sparse-attention transformers.

2.2 Why Sparsity Changes the Design Problem

2.2.1 The FLOP-to-Bandwidth Mismatch

Dense matrix multiplication is efficient because it has high operational intensity: for an $M \times K \times N$ General Matrix Multiply (GEMM), many Floating Point Operations (FLOPs) are performed for every byte of data loaded from memory, with each operand reused across the N output columns. Sparse matrix multiplication breaks this reuse pattern. For a CSR SpMM with NNZ nonzeros, the number of useful MACs per byte of CSR index data fetched is

$$\mathrm{OI} = \frac{2 \cdot \mathrm{NNZ} \cdot N}{\mathrm{NNZ} \cdot (4+4) + (M+1) \cdot 4 + \mathrm{NNZ} \cdot 4 \cdot N},$$

where the denominator accounts for `colidx` (4 bytes per entry), `values` (4 bytes per entry), `rowptr` (4 bytes per pointer), and the dense B segment reads (4 bytes per column per nonzero). For small N and low density, the operational intensity is significantly below the roofline knee of typical on-chip BRAM systems [5], indicating that SpMM is memory-bandwidth-limited rather than compute-limited on most platforms.

2.2.2 Irregular Access and Control-Flow Overhead

Beyond raw bandwidth, SpMM imposes control overhead that dense accelerators are not designed to handle. For each nonzero a_{ij}, the hardware must:

1. read the column index j from `colidx`;
2. compute the address of the relevant row of B as B_base $+ j \cdot N$;
3. read the N values of $B[j, :]$; and
4. accumulate $a_{ij} \cdot B[j, n]$ into $C[i, n]$ for each column n.

Steps (1) and (2) introduce indirection, often called "pointer-chasing": the next address depends on a value that has just been read. This breaks prefetching, and on an in-order core such as PicoRV32 each stall is exposed directly in the cycle count, so control overhead rather than arithmetic becomes the primary bottleneck. Figure 2.2 contrasts the predictable stride of a dense traversal with the indirect, data-dependent address sequence that CSR SpMM produces.

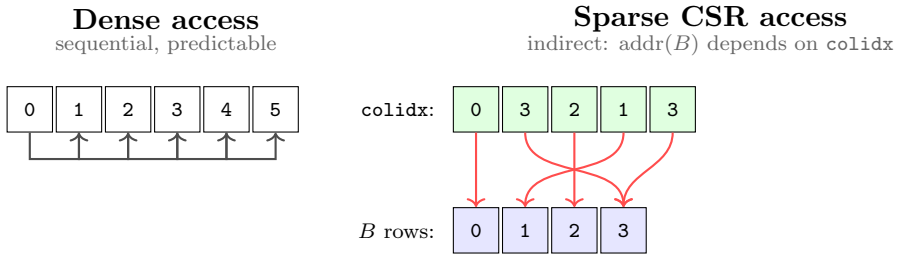

FIGURE 2.2: Memory access patterns in dense versus sparse CSR traversal. Dense access strides sequentially through contiguous addresses, so a prefetcher can fetch ahead. CSR access loads a column index from `colidx` first, then uses that value as an indirect pointer into B; the next address cannot be known until the previous load completes, which breaks prefetching and produces data-dependent cache misses.

2.3 Sparse Matrix Storage Formats

Several sparse matrix storage formats have been proposed; the choice of format strongly affects the hardware control structure. Table 2.1 summarises the most relevant formats and their suitability for hardware acceleration.

Format	Acronym	Description	Hardware suitability
Coordinate list	COO	Stores (row, col, val) triplets unsorted or sorted by row. Simple to generate; requires sorting for efficient row-wise access.	High overhead per nonzero; poor for row-streaming hardware.
Compressed Sparse Row	CSR	Three arrays: `rowptr` (M+1 entries), `colidx` (NNZ), `values` (NNZ). Row bounds are contiguous.	Excellent for row-wise traversal; natural hardware FSM mapping.
ELLPACK	ELL	Pads each row to the maximum NNZ per row; stores in column-major order.	Good for regular sparsity; wastes storage and bandwidth when rows vary widely.
Block Compressed Sparse Row	BSR	Groups nonzeros into dense $r \times c$ blocks stored in CSR form.	High throughput on block-sparse matrices; unsuitable for unstructured sparsity.
Compressed Sparse Column	CSC	Transpose of CSR; columns are compressed.	Natural for column-wise access patterns; awkward for row-oriented output.

TABLE 2.1: Comparison of common sparse matrix storage formats and their suitability for hardware acceleration.

2.3.1 Why CSR Was Chosen

Figure 2.3 shows the CSR encoding of a small 4×4 sparse matrix. The three arrays `rowptr`, `colidx`, and `values` together record exactly which nonzeros exist and where, with no storage spent on the zero entries.

CSR was selected for this project because it maps directly onto the natural execution order of row-wise SpMM. Gustavson's algorithm for sparse matrix multiplication [6] established that row-wise traversal of CSR is asymptotically optimal for output-row-oriented computation, and that the row pointer array provides $O(1)$ access to the start of each row's nonzero sequence. Bell and Garland [7] demonstrated that CSR remains the most widely supported format for both CPU and GPU SpMV/SpMM implementations precisely because its memory layout matches the natural iteration over output rows.

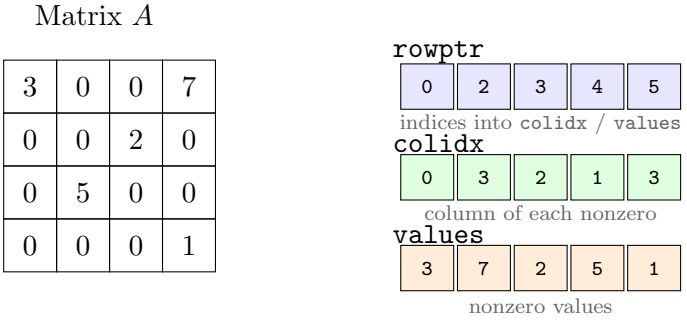

FIGURE 2.3: CSR encoding of a 4×4 sparse matrix with five nonzeros. `rowptr`[i] is the index of the first nonzero in row i; row i owns entries `rowptr`[i] through `rowptr`[i+1]−1 in both `colidx` and `values`. `rowptr`[M]=NNZ by convention.

For a hardware FSM, the CSR structure offers a critical advantage: the bounds of each row's nonzero segment are available with a single pointer read, and the entire nonzero segment can be streamed sequentially from memory. There is no need for global index sorting, block alignment padding, or two-dimensional address translation. The firmware preparation cost is also low: generating CSR from a list of nonzeros requires a single pass to count per-row nonzeros and a second pass to write values and indices.

2.4 SpMM Execution with CSR

The previous section explained how CSR encodes a sparse matrix. This section explains how a sparse–dense matrix multiply actually uses that encoding. The distinction matters for hardware, because the same three arrays that describe the storage of A also drive the address generation of the accelerator.

2.4.1 Problem Statement

Given a sparse matrix $A \in \mathbb{R}^{M \times K}$, a dense matrix $B \in \mathbb{R}^{K \times N}$, and an output $C \in \mathbb{R}^{M \times N}$, the product $C = AB$ is defined element-wise by

$$C[i, n] \;=\; \sum_{j=0}^{K-1} A[i, j] \cdot B[j, n].$$

For a dense A the inner sum performs K multiplies per output element, the vast majority of which are zero when A is sparse. CSR removes those zeros from the

loop entirely: only the nonzero entries of each row of A are enumerated, and each one contributes a scaled copy of a single row of B to the corresponding row of C.

2.4.2 A Concrete Example

Figure 2.4 shows a 4×4 sparse A matrix, multiplied by a 4×3 dense B to produce a 4×3 output C. A has five nonzeros, so only five sparse–dense interactions occur in the entire multiply. Each one takes a single nonzero of A and a single row of B (the row whose index equals the column index of the nonzero) and produces a scaled row that is accumulated into the matching row of C. The highlighted entry $A[0,3] = 7$ picks row 3 of B, multiplies it by 7, and contributes $(70, 77, 84)$ to row 0 of C; $A[0,0] = 3$ contributes a further $(3, 6, 9)$ from row 0 of B, giving the final $C[0,:] = (73, 83, 93)$.

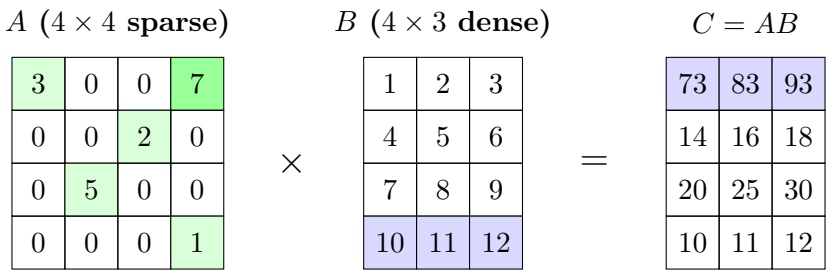

FIGURE 2.4: A small sparse–dense matrix multiply $C = AB$. Shaded cells of A are its five nonzero entries; the darker cell $A[0,3] = 7$ is highlighted as a worked example. Each nonzero of A scales the row of B whose index equals its column and accumulates that scaled row into the matching row of C.

2.4.3 Row-Wise CSR Execution

In CSR form, the nonzeros of A are stored in row-major order as described in Section 2.3. The computation of $C = AB$ iterates over each output row and then over each nonzero in that row:

1. For each output row $i = 0, 1, \ldots, M-1$, look up the row bounds $p_{\text{start}} =$ `rowptr`$[i]$ and $p_{\text{end}} =$ `rowptr`$[i+1]$.
2. For each $p \in [p_{\text{start}}, p_{\text{end}})$, read the column $j =$ `colidx`$[p]$ and the scalar $a =$ `values`$[p]$, fetch the dense row $B[j,:]$, and accumulate $C[i,:] \mathrel{+}= a \cdot B[j,:]$.

The inner update is a scalar-times-vector operation across the N output columns, so one nonzero of A produces N useful multiplies in parallel, and the sparse dimension K never appears as a loop variable. The parallelism in the hardware comes from the dense dimension, where the work is regular, rather than the sparse dimension, where it is not.

Table 2.2 traces the two iterations that compute row $i = 0$ of the example. The bounds `rowptr`$[0] = 0$ and `rowptr`$[1] = 2$ select two nonzeros from `colidx` and `values`. The first iteration uses `values`$[0] = 3$ and `colidx`$[0] = 0$ to fetch $B[0, :]$ and accumulate $3 \cdot B[0, :]$ into $C[0, :]$. The second iteration uses `values`$[1] = 7$ and `colidx`$[1] = 3$ to fetch $B[3, :]$ and accumulate $7 \cdot B[3, :]$. The two partial updates sum to $C[0, :] = (73, 83, 93)$, agreeing with Figure 2.4.

p	`values`[p]	`colidx`[p]	$B[j, :]$	$a \times B[j, :]$	$C[0, :]$ after update
0	3	0	$(1, 2, 3)$	$(3, 6, 9)$	$(3, 6, 9)$
1	7	3	$(10, 11, 12)$	$(70, 77, 84)$	$(73, 83, 93)$

TABLE 2.2: Walk-through of the row-wise CSR execution for output row $i = 0$ of Figure 2.4. Each iteration reads one nonzero of A through the CSR arrays, fetches the corresponding row of B, and accumulates the scalar-scaled vector into $C[0, :]$.

2.4.4 Why This Maps Well to Hardware

The execution model has three properties that suit a small FPGA accelerator:

1. *Address generation from three arrays.* `rowptr` supplies the loop bounds for each output row, `colidx` supplies the row index of B to fetch, and `values` supplies the scalar multiplier. No global index sort, block alignment, or two-dimensional address translation is required.

2. *Data-parallel inner update.* The scalar-vector update maps onto an array of TN parallel MAC units, each handling one column of the output tile, so the MAC array sees regular work even when the outer traversal is irregular.

3. *Amortised irregular cost.* Each operand fetched from `colidx` and `values` drives N multiplies in the output, so the cost of the indirect CSR lookup is spread over all N output columns.

The control FSM described in Chapter 4 implements exactly this loop structure, with the three CSR arrays feeding the address generation logic and the row of B forming the operand tile that is multiplied against each nonzero scalar.

2.5 Prior Work on SpMM Accelerators

2.5.1 OuterSPACE

OuterSPACE [8] targets the SpGEMM (sparse–sparse matrix multiply) problem using an outer-product decomposition. Rather than traversing CSR rows serially, it computes partial products for each nonzero column of A and accumulates them into a merge tree, achieving high throughput by parallelising across columns of A and using dedicated on-chip accumulators.

2.5.2 SpArch

SpArch [9] also targets sparse–sparse computation, focusing on the merger phase that reduces partial products to a final output. It introduces a condenser that reorganises partial products to maximise row-merger bandwidth, and shows that the merger step, not the multiplication step, typically dominates sparse–sparse computation time.

2.5.3 ExTensor

ExTensor [10] generalises sparse computation to higher-order tensors and introduces intersection hardware that skips pairs of zero operands before they reach the multiplier array, avoiding the memory read entirely when a nonzero pair is absent. CSR provides a weaker form of this by construction: the column-index array only points to nonzero entries, and zero rows of A produce no traversal steps at all.

2.5.4 SIGMA

SIGMA [11] targets the sparse-weight GEMM that arises during DNN training after unstructured pruning. It uses a flexible *Forwarding Adder Network* (FAN) as a

reduction tree whose connectivity is reconfigured at runtime to match the nonzero pattern of the weight matrix, showing that hardware-reconfigurable reduction topologies can absorb the irregular sparsity of training-time gradients.

2.5.5 Positioning This Work

Taken together, the four designs span complementary regions of the sparse-acceleration space. OuterSPACE and SpArch prioritise high-throughput sparse–sparse accumulation, with SpArch specifically attacking the merger phase. ExTensor emphasises operand intersection and zero-pair skipping. SIGMA targets reconfigurable reduction for sparse DNN training. Each assumes a throughput regime, scheduling mechanism, or memory hierarchy that is not available on a Cyclone V FPGA. This project instead targets the simpler but practically relevant CSR SpMM case on a resource-constrained FPGA SoC, aligning the storage format, datapath, and quantisation strategy with the dominant cost of the workload.

2.6 Hardware Platform Alternatives

General-purpose CPUs handle SpMM inefficiently because the irregular access pattern defeats prefetching and out-of-order speculation, and on a small in-order soft-core such as PicoRV32 there is no data cache at all. GPUs achieve high SpMM throughput through massive parallelism [7] but require 100–400 W and a substantial runtime stack, which rules them out for an embedded SoC. A custom ASIC would deliver the best throughput and power, but the development cost and fabrication lead time are impractical for a final-year project.

FPGAs provide a practical middle ground: custom datapath parallelism, flexible on-chip memory organisation, and far lower power than a GPU [12]. The Cyclone V on the DE1-SoC supplies M10K BRAM blocks, DSP multiplier-accumulator blocks, and configurable logic that can be combined into a bespoke CSR SpMM datapath with a memory interface tailored to the access pattern, which is a practical choice for prototyping an accelerator architecture before any ASIC commitment.

2.7 INT8 Quantisation

2.7.1 Motivation

Quantisation reduces the numerical precision of stored values, decreasing memory footprint and potentially increasing arithmetic throughput [13, 14]. For SpMM specifically, quantisation acts directly on the bottleneck: an un-packed 32-bit word carries one operand, whereas INT8 packing places four operands in the same word. For a memory-bound kernel this reduces the number of RAM reads needed to fill an operand tile by the same factor, which is the dominant reason INT8 is attractive for SpMM. The concrete cycle-level implications on the accelerator datapath are developed in Chapter 4.

2.7.2 Symmetric Linear Quantisation

The project adopts runtime symmetric linear quantisation. Given a vector $\mathbf{v}$ of full-precision values, the maximum absolute value $v_{\max} = \max_i |v_i|$ is computed, and each value is mapped to a signed 8-bit integer by

$$q_i = \text{clip}\left(\text{round}\left(\frac{v_i}{v_{\max}} \cdot 127\right), -127, 127\right), \tag{2.1}$$

where the scale factor is $s = v_{\max}/127$. Reconstruction uses $\hat{v}_i = q_i \cdot s$, so the quantisation error is bounded by $|e_i| \leq s/2$. Symmetric quantisation was preferred over asymmetric because it eliminates the need for a zero-point offset in the inner MAC loop, simplifying the hardware accumulation logic.

2.7.3 Accumulation Precision

INT8 accumulation into INT8 is insufficient for most practical workloads: a sum of K products of INT8 values each up to ± 127 can reach $\pm 127^2 \cdot K = \pm 16{,}129K$, which overflows INT8 and INT16 even for moderate K. The present design therefore accumulates into INT32, which can represent sums of up to $2^{31}/16{,}129 \approx 133{,}000$ elements before overflow. This matches the widely adopted practice of INT8×INT8 with INT32 accumulation as described by Jacob et al. [13].

2.8 Roofline Analysis and Design Implications

The roofline model [5] characterises whether a kernel is compute-bound (throughput limited by the peak arithmetic rate) or memory-bound (throughput limited by available bandwidth). The position of a kernel on the roofline is determined by its *arithmetic intensity*: the ratio of arithmetic operations to bytes of data movement. A kernel whose intensity places it to the left of the roofline knee is memory-bound, and its achievable throughput scales linearly with bandwidth; one placed to the right is compute-bound, and its throughput is capped by the peak arithmetic rate.

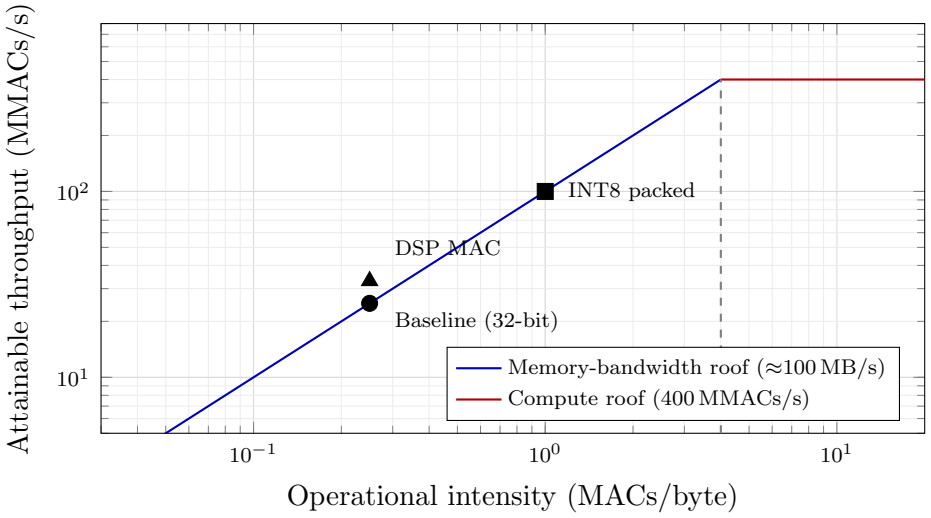

FIGURE 2.5: Roofline interpretation of the three accelerator stages on the Cyclone V platform. The baseline and DSP-enhanced designs sit on the memory-bandwidth roof: arithmetic throughput is capped by operand fetch, so widening the MAC array yields only a small gain. INT8 packing raises the operational intensity by a factor of four (four operands per 32-bit word), shifting the design rightward and upward along the bandwidth roof towards the compute-bound region.

SpMM sits firmly in the memory-bound regime for the reasons developed in Section 2.2: low operand reuse and indirect access keep the arithmetic intensity well below the knee. The practical implication for accelerator design is that widening the MAC array yields only a small gain unless operand supply is widened alongside it. Quantisation addresses this directly by increasing the number of useful operands packed into each memory transaction, moving the design's operating point along the bandwidth roof towards the compute-bound plateau. A numerical instantiation of this argument for the present $TN = 8$ accelerator is given in Chapter 4, and Figure 2.5 illustrates the qualitative shift that INT8 packing produces.

Chapter 3

System Design and RISC-V Platform

This chapter describes the system that hosts the SpMM accelerator: what the SoC looks like as a whole, why a soft RISC-V core was chosen as the host, how memory and the bus are organised, and how firmware drives the accelerator through a small bank of memory-mapped registers. The accelerator itself is treated as a black box here; its internal datapath and optimisation are the subject of Chapter 4.

3.1 System Overview

The SoC is synthesised entirely into FPGA fabric on the Terasic DE1-SoC development board [15]. The hard ARM Cortex-A9 on the HPS side of the device is unused, so the whole design is self-contained in programmable logic. Every block in the benchmark path lives on-chip: firmware, CSR arrays, the dense matrix B, and the output matrix C all reside in a shared 64 KB dual-port block RAM. The design runs from the 50 MHz on-board oscillator.

Keeping the design inside the FPGA has three consequences that shape the rest of this chapter. First, there is no AXI bridge, no Linux boot, and no HPS memory management to reason about, so cycle counts taken on the PicoRV32 core are directly comparable with cycle counts taken on the accelerator. Second, the problem sizes are bounded by on-chip capacity rather than by external DRAM bandwidth, which removes one variable from the comparison at the cost of limiting absolute problem size. Third, the software and hardware paths share the same

memory, which makes the dual-port arrangement described in Section 3.4 the central architectural decision of the platform.

Figure 3.1 shows the top-level block diagram. The PicoRV32 CPU, the SpMM accelerator, UART, and GPIO all hang off a simple interconnect that routes requests to the BRAM or to one of the memory-mapped peripheral regions.

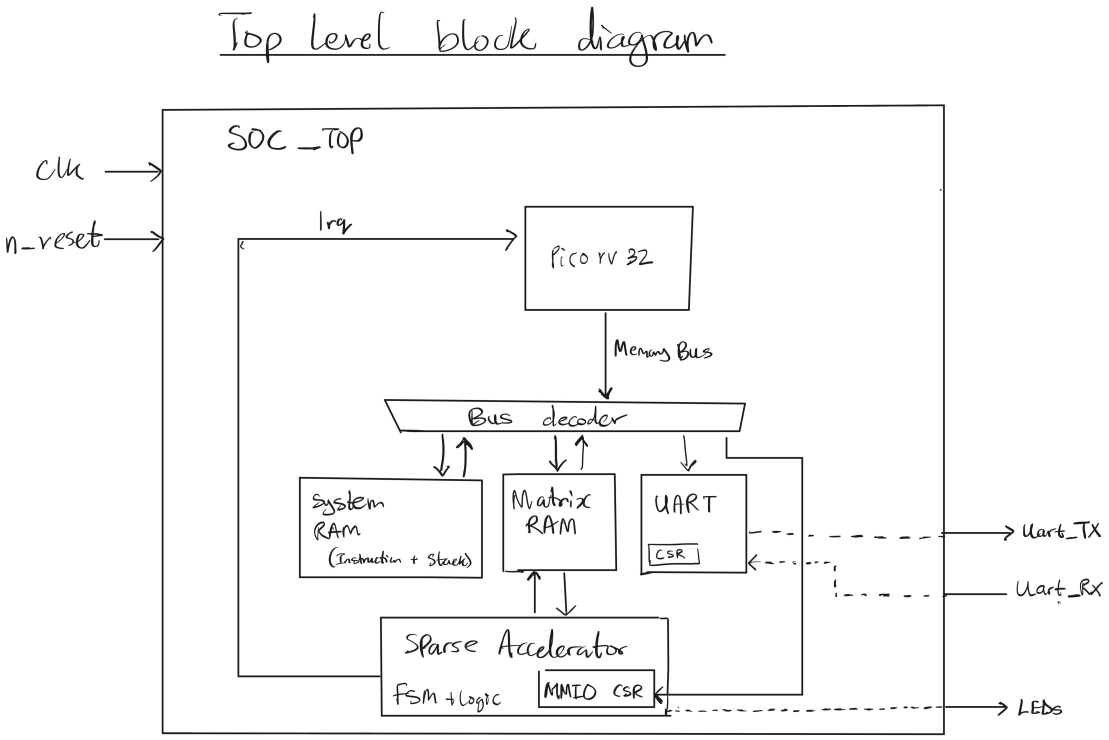

FIGURE 3.1: Top-level RISC-V SoC block diagram.

3.2 Why a Soft RISC-V SoC

The host processor is PicoRV32 [16], a compact RV32IM soft-core implementing the RISC-V base integer instruction set with the hardware integer multiply extension [17]. Three practical reasons drove the choice.

PicoRV32 is small, roughly 700 to 1000 ALMs depending on configuration, which leaves the majority of the Cyclone V fabric available for the accelerator and the 64 KB BRAM. It exposes the `rdcycle` performance counter CSR natively, so all benchmark timing in this report is cycle-accurate without needing an external timer or off-chip instrumentation. And its memory bus is a plain valid/ready handshake

with address, data, and write-enable signals, which drops straight onto the SoC interconnect without an AXI adapter.

The important caveat for interpreting the results later in this report is that PicoRV32 is in-order and non-pipelined. It issues one instruction at a time and stalls fully on every load. For a synchronous BRAM access, the CPU waits one cycle after presenting the address before the data is returned. This makes the software baseline slower than a modern superscalar CPU would be, which means the reported accelerator speedup is larger than it would be against, say, a Cortex-A class core. The comparison is still meaningful because the goal is to quantify the benefit of custom SpMM hardware against the same RV32IM ISA, not to claim absolute superiority over a desktop processor.

The M extension matters for the same reason. The software SpMM reference uses hardware `MUL` instructions for the inner multiply-accumulate. Without it, PicoRV32 would fall back to a software multiply routine that is typically ten to thirty times slower, which would artificially inflate every speedup number in Chapter 4. The M extension is therefore a prerequisite for a credible software baseline.

3.3 Target FPGA Platform

The Cyclone V 5CSEMA5F31C6 on the DE1-SoC provides the resources summarised in Table 3.1 [18]. The estimated utilisation column reports the design's approximate footprint; final synthesis numbers are pending completion of the Quartus timing closure study.

Resource	Total available	Used (estimated)
Adaptive Logic Modules (ALMs)	32,070	≈5,000 (16%)
Logic registers	128,280	≈8,000 (6%)
M10K memory blocks (10 Kbit each)	397	≈20 (5%)
DSP 18×18 multiplier blocks	87	≈8 (9%)
I/O pins	457	—

Table 3.1: Cyclone V 5CSEMA5F31C6 resource availability and estimated design utilisation.

Two resource classes are load-bearing for this design. The M10K blocks each provide 10 Kbits (roughly 1.25 KB) of configurable on-chip memory [19]. In true-dual-port mode with 32-bit-wide ports, each M10K holds 256 words, so the 64 KB shared BRAM needs around 16 M10Ks arranged as a cascade. That leaves plenty

of headroom for any additional buffers the accelerator might want to add later. The DSP blocks provide 18×18-bit signed multipliers with a dedicated accumulator register [20], which maps cleanly onto INT32 MAC operations and, with the packed-byte treatment described in Chapter 4, onto INT8 MACs as well.

3.4 Memory and Interconnect Architecture

3.4.1 Bus Protocol

The SoC interconnect uses the native PicoRV32 memory bus: a valid/ready handshake with a 32-bit address bus, a 32-bit bidirectional data bus, a 4-bit byte-enable signal, and a read/write indicator. Everything is synchronous to the 50 MHz system clock. Access latency is one cycle for BRAM (using the BRAM's registered output) and combinatorial for MMIO registers.

3.4.2 Dual-Port BRAM Organisation

The 64 KB on-chip RAM is implemented as an Altera `altsyncram` true-dual-port memory with 32-bit-wide ports. Port A belongs to the CPU and supports byte-granular writes through a 4-bit byte-enable mask. Port B belongs to the accelerator and uses 32-bit word-granular access. Figure 3.2 shows the arrangement.

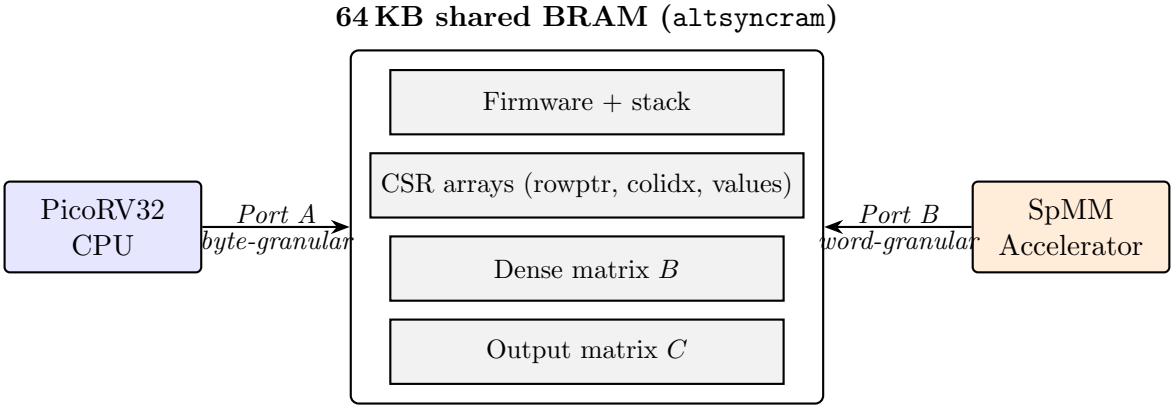

FIGURE 3.2: Dual-port BRAM organisation. The CPU and accelerator drive independent ports on the same memory array, so they can read or write different addresses on the same cycle without arbitration.

Because Port A and Port B address the same underlying array, two accesses that hit the same word on the same cycle produce undefined results. The firmware never

writes to a buffer that the accelerator is actively reading, and it never reads C until the accelerator has signalled completion. Apart from that constraint, the two sides are free to operate concurrently: during a benchmark run, for example, the CPU can stream the previous output over UART on Port A while the accelerator populates the next output on Port B.

3.4.3 Address Space

Table 3.2 summarises the full memory map, and Figure 3.3 shows the same information as a vertical strip for quick reference.

Address range	Peripheral	Purpose
`0x0000_0000--0x0000_FFFF`	On-chip RAM (64 KB)	Firmware code, stack, CSR arrays (`rowptr`, `colidx`, `values`), dense matrix B, output matrix C
`0x1000_0000--0x1000_00FF`	SpMM accelerator MMIO	Configuration registers, control, and status
`0x2000_0000--0x2000_000F`	GPIO	LED output, switch and button input, seven-segment display
`0x2000_0100--0x2000_01FF`	UART (`simpleuart`)	Serial output for UART benchmark reporting at 115200 baud

TABLE 3.2: System memory map of the implemented RISC-V SoC.

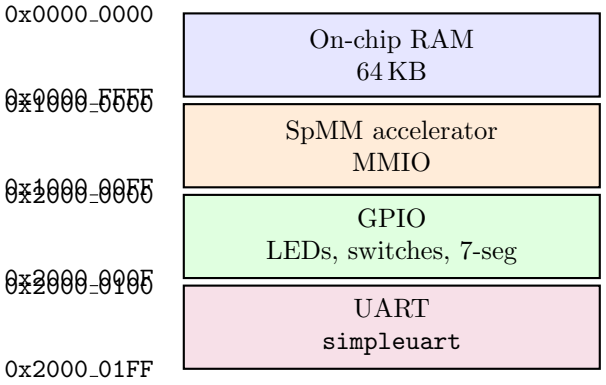

FIGURE 3.3: System memory map as a vertical address strip. Each region occupies a distinct high-order address decode so that routing inside the interconnect is a simple comparison on the top byte of the address.

3.5 Accelerator Programming Interface

The accelerator is controlled entirely through memory-mapped registers at `0x1000_0000`–`0x1000_002C`. Firmware writes dimensions and base addresses, pulses the start bit, polls the status register, and reads the output once the done flag rises. There is no DMA engine, no interrupt controller, and no driver layer; the whole interface is suitable for bare-metal firmware. Table 3.3 lists the register set.

Offset	Name	Description
`0x00`	`CTRL`	Control register: bit 0 = `start` (self-clearing pulse), bit 1 = `clear_done`, bit 2 = `irq_en` (reserved; polling used), bit 3 = `relu_en`, bit 4 = `int8_mode`
`0x04`	`STATUS`	Status register: bit 0 = `busy`, bit 1 = `done`
`0x08`	`M`	Number of rows in A and C
`0x0C`	`N`	Number of columns in B and C (dense width)
`0x10`	`K`	Number of columns in A / rows in B
`0x14`	`A_VAL_BASE`	Base address of CSR `values` array
`0x18`	`A_ROW_BASE`	Base address of CSR `rowptr` array
`0x1C`	`A_COL_BASE`	Base address of CSR `colidx` array
`0x20`	`B_BASE`	Base address of dense matrix B (row-major)
`0x24`	`C_BASE`	Base address of output matrix C (row-major)
`0x28`	`NNZ`	Total number of nonzeros in A

TABLE 3.3: SpMM accelerator MMIO register map. All registers are 32-bit word-aligned. Base addresses are byte addresses into the shared 64 KB RAM.

The `start` bit behaves as a one-cycle pulse: the CPU writes a 1, the accelerator captures it, and the bit self-clears so the CPU never has to clear it explicitly. `STATUS.done` is set when the last output row has been written back and remains high until the CPU acknowledges it by writing `CTRL.clear_done`. The interrupt-enable bit exists in the hardware but is currently unused; the firmware polls `STATUS` instead.

3.6 System Operation

Figure 3.4 shows the handshake between firmware and accelerator for a single SpMM run. The CPU stages all inputs in BRAM, writes the MMIO configuration, pulses `start`, and then polls the status register. The accelerator runs asynchronously on Port B of the BRAM until it has written the last element of C, at which point it raises `done`. The CPU clears the flag and proceeds to read C.

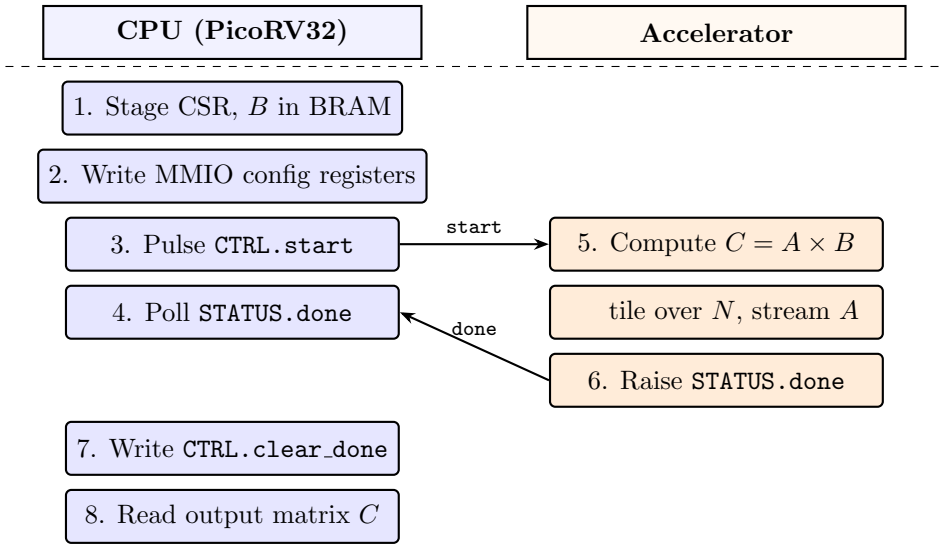

FIGURE 3.4: System operation sequence for a single SpMM run. Steps 1–4 and 7–8 execute on the CPU; steps 5–6 execute on the accelerator concurrently with the CPU's polling loop. The two lanes synchronise only through the `start` pulse and the `done` flag.

In more detail, the accelerator is specified to:

1. read the matrix dimensions M, N, K, and NNZ from the MMIO registers when `start` is pulsed;
2. interpret A in CSR form using the three base-address registers (`A_VAL_BASE`, `A_ROW_BASE`, `A_COL_BASE`);
3. read the dense matrix B from shared RAM in row-major layout;
4. compute $C = A \times B$ using tiled parallel MAC operations across the dense-output dimension;
5. optionally apply element-wise ReLU to each output element before writeback when `relu_en` is asserted; and
6. signal completion by asserting `STATUS.done` and releasing `STATUS.busy`.

When `CTRL.int8_mode` is set, the accelerator reinterprets the `values` array and matrix B as packed signed bytes, sign-extending each byte to 32 bits before multiplication. The packed-byte layout and its effect on BRAM bandwidth is analysed in Chapter 4.

3.7 Design Constraints and Implications

Four constraints shaped the platform decisions described above.

No external memory interface. All benchmark data lives in on-chip BRAM. This removes the SDRAM or DDR controller and the associated latency from the comparison, at the cost of bounding the problem size. In practice the firmware can size problems up to roughly $M = K = 64$, $N = 32$, depending on NNZ, before the combined CSR, B, and C footprint exceeds the 64 KB budget.

Simple MMIO control. The accelerator must be usable from bare-metal firmware with no OS, no DMA engine, and no driver stack. Register writes and a polling loop are enough, which is why the interface is intentionally small.

Dense-dimension tiling with $TN = 8$. The accelerator exploits parallelism across output columns in fixed-width tiles. Eight was chosen to provide meaningful parallelism (eight simultaneous MACs) without exceeding the Cyclone V DSP and BRAM routing limits at 50 MHz. Chapter 4 justifies this choice against the roofline model.

Reproducible cycle-accurate benchmarking. The firmware benchmark suite is deterministic (fixed PRNG seeds), self-verifying (element-by-element comparison against a software reference), and timed using PicoRV32's `rdcycle` CSR. Generating data at runtime rather than loading it from external storage was a direct consequence of the on-chip-only constraint.

Together these constraints explain the deliberate simplicity of the polling completion model, the modest tile width, and the on-chip data layout that the remainder of the report relies on.

Chapter 4

Accelerator Design

4.1 Design Philosophy and Architecture

The accelerator was shaped by the execution structure of CSR SpMM rather than adapted from a generic dense-matrix template. CSR SpMM has three distinct computational phases (row-bound loading, nonzero streaming, and dense tile accumulation), and each phase has a different memory and control-flow character. Any hardware design that treats them as a single monolithic pipeline inevitably over-engineers one phase and underserves another, so the architecture instead separates concerns explicitly: a control finite state machine (`accel_ctrl`) walks the CSR structure and sequences RAM accesses, while a datapath (`accel_datapath`) performs the parallel multiply–accumulate and manages the local tile buffers. This split follows standard digital design practice [21] and, more importantly, allows each half of the accelerator to be verified independently in simulation before integration.

The top-level module (`accel_top`) wraps these two blocks together with an MMIO front-end that decodes the PicoRV32 memory bus and exposes the configuration interface of Table 3.3. The accelerator reaches shared memory through Port B of the dual-port BRAM, a 32-bit word-granular port with one cycle of registered read latency. All RAM accesses are therefore scheduled as paired issue/consume phases separated by exactly one clock cycle, which gives the controller a deterministic latency model and removes the need for a stall/ready handshake.

4.2 Baseline Datapath

The datapath is parameterised by a tile width $TN = 8$ (the number of output columns processed in parallel), a maximum row count $M_{\max} = 64$, and a 32-bit signed operand width. Its behaviour depends on two local buffers whose sizing is critical to performance. The first, `B_Seg`, is an eight-element array of signed 32-bit values that holds the current row segment of dense matrix B: for every nonzero a_{ij} the controller fills `B_Seg` with $B[j,\, c_0 : c_0{+}TN]$ before enabling the MAC array. The second, `C_Tile`, is an $M_{\max} \times TN$ array of signed 32-bit partial sums that accumulates the output tile for all rows of C simultaneously within the current tile column $[c_0, c_0{+}TN)$. Using a flat two-dimensional accumulator rather than per-row registers avoids writeback/restore cycles when nonzeros from different rows touch the same tile column, which is the common case in realistically distributed sparse matrices. The C-tile is implemented as a 512-register fabric array rather than BRAM; on the Cyclone V each ALM carries two registers, so the tile consumes roughly 256 ALMs, well within the budget of the target device.

For each nonzero $a = A[i,k]$ the controller asserts `mac_en`, `mac_row` $= i$, and `mac_a` $= a$, and the datapath issues eight simultaneous signed multiply–accumulate operations:

$$\texttt{C\_Tile}[i][c] \;\mathrel{+}=\; \texttt{mac\_a} \times \texttt{B\_Seg}[c] \qquad c = 0, 1, \ldots, TN{-}1.$$

This is the key mapping that gives the accelerator its parallelism: a single nonzero in A interacts with an entire row segment of B, so one issue cycle produces TN output updates. The parallelism is across the dense output dimension, where the workload is regular, rather than along the sparse dimension, where it is not. When all rows have been processed in the current tile column, the C-tile is streamed back to shared RAM row by row; if `CTRL.relu_en` is asserted, an element-wise $\max(0, \cdot)$ is applied to each value on writeback, making the accelerator directly usable as the fully-connected layer of a neural network inference pipeline.

4.3 Control FSM

The control FSM is the most intricate component of the design. It drives all address generation, RAM sequencing, nonzero traversal, and tile management, and cycles through the phases listed in Table 4.1; the sub-phase indices within `ROW_LOAD` and

NZ_FETCH exist specifically to cover the one-cycle registered RAM latency. The simplified state-level view is shown in Figure 4.1.

State	Action
IDLE	Wait for `start` pulse. Load M, N, K, NNZ, and base addresses from the MMIO register snapshot.
INIT_TILE_CLEAR	Clear all entries of `C_Tile` to zero in preparation for the first tile column (one row per cycle, $M_{\max}$ cycles).
ROW_LOAD	Load the CSR row bounds for the current row i: issue reads for `rowptr[i]` and `rowptr[i+1]`; handle the registered RAM output latency using sub-phases `ROW_PHASE_0/1/2`.
NZ_FETCH	For each nonzero in `[rowptr[i], rowptr[i+1])`: read `values[p]` and `colidx[p]` and compute the starting address of $B[\texttt{col}, :]$ for the current tile. Sub-phases `NZ_PHASE_0`–`NZ_PHASE_4` handle RAM latency and address arithmetic.
MAC	Fill `B_Seg` with TN consecutive values from B (one per clock in INT32 mode, two reads total in INT8 mode), then assert `mac_en` for one cycle to update `C_Tile`.
NEXT_ROW	Increment the row index. If more rows remain in this tile column, loop to `ROW_LOAD`; otherwise proceed to `WRITE_TILE`.
WRITE_TILE	Write all $M \times TN$ entries of `C_Tile` back to shared RAM (with optional ReLU). Advance the tile column offset by TN; if more tile columns remain, clear `C_Tile` and loop to `ROW_LOAD`, otherwise proceed to `DONE`.
DONE	Assert `STATUS.done` and release `STATUS.busy`; return to `IDLE` when `clear_done` is asserted by the CPU.

TABLE 4.1: Accelerator FSM states and their actions.

For a matrix with M rows, NNZ nonzeros, and dense width N (processed in N/TN tile columns), the approximate cycle budget per tile column is

$$T_{\text{tile}} \approx M_{\max} + M \cdot C_{\text{row}} + \text{NNZ} \cdot (C_{\text{nz}} + TN) + M \cdot TN \cdot C_{\text{wb}},$$

with $C_{\text{row}} \approx 5$ cycles per row-bound load, $C_{\text{nz}} \approx 5$ cycles per nonzero fetch, and $C_{\text{wb}} \approx 1$ cycle per output writeback. At $TN = 8$ the nonzero fetch and B-segment fill together contribute roughly $5 + 8 = 13$ cycles per nonzero, and the reader will

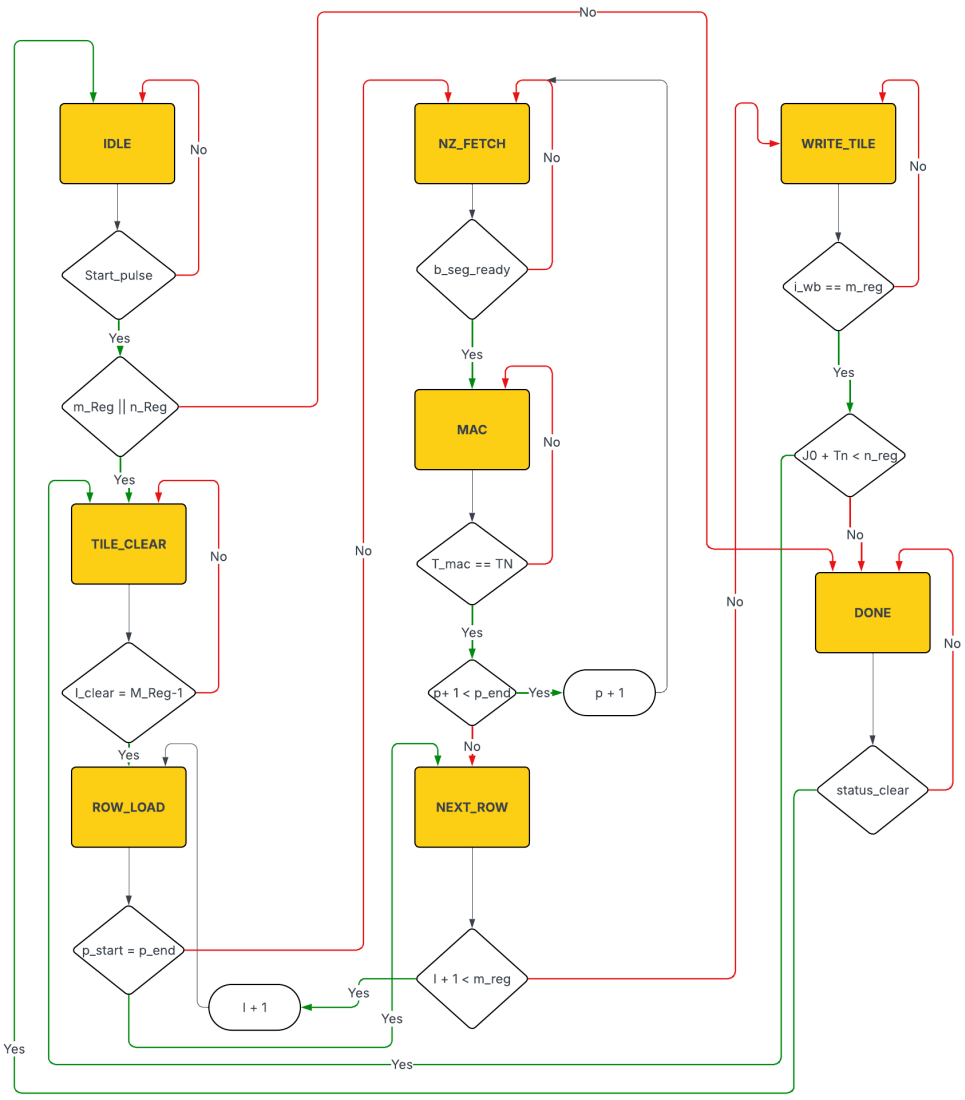

FIGURE 4.1: Simplified accelerator finite state machine. Each major phase corresponds to one row in Table 4.1. Sub-phase transitions within `ROW_LOAD` and `NZ_FETCH` handle the one-cycle registered RAM output latency.

see directly how shrinking the TN-dependent term is what the INT8 optimisation later attacks.

4.4 The Choice of Tile Width

The tile width TN sets the degree of parallelism in the dense dimension, and its selection is a balance between arithmetic throughput, register array size, memory-access burden, and routing feasibility on the target device. Table 4.2 summarises the options. At $TN = 1$ the accelerator degenerates to a sequential MAC with no parallel benefit; $TN = 4$ offers modest parallelism at the price of halving the reuse of each fetched B segment; $TN = 16$ or $TN = 32$ doubles or quadruples the register array and the number of required DSP blocks, at which point both routing congestion and timing closure become the limiting factor on a Cyclone V that already carries a CPU, BRAM, UART, and GPIO. A tile width of eight maps cleanly onto eight DSP multiply–accumulate blocks, keeps the 512-register C-tile within practical limits, and covers the smallest benchmark width ($N = 8$) in a single tile column so no case in the suite is penalised by tiling overhead. It was also the largest width that consistently closed timing at 50 MHz in preliminary synthesis, which settled the choice for the final design.

TN	MAC lanes	Acc. registers	B-seg reads/NZ	DSP estimate	Notes
1	1	64	1	1	Minimal resources; no parallelism; effectively sequential.
4	4	256	4	4	Moderate parallelism; reduced register pressure.
8	8	512	8	8	**Design point.** Good balance; DSP-mappable; 512-reg array feasible.
16	16	1024	16	16	Doubles resources; increases routing congestion; timing harder to close.
32	32	2048	32	32	Likely infeasible on Cyclone V; 2048-reg array exceeds practical limits.

TABLE 4.2: Resource and performance implications of different tile widths TN. The accelerator implements $TN = 8$.

4.5 Algorithmic Decomposition: Row-Wise CSR Traversal

The first and most consequential design decision was algorithmic: process CSR rows serially and tile only the dense dimension. Alternatives such as outer-product decomposition (as in OuterSPACE [8]) or intersection-based skipping (as in ExTensor [10]) can deliver higher throughput in certain regimes, but they do so by introducing index-merge networks, multi-port accumulation, and in some cases content-addressable storage, none of which are feasible inside the SoC budget of this project. Row-wise CSR traversal keeps the FSM tractable enough to verify by hand, keeps the BRAM Port B access pattern regular so no bank conflicts arise, and keeps the C-tile writeback purely sequential with no merge logic. The simplicity pays for itself in both correctness confidence and timing-closure margin.

4.6 Memory Optimisation: the B-Segment Bottleneck

The conceptual roofline interpretation introduced in Section 2.8 can now be made concrete for this platform. Operating at 50 MHz with a $TN = 8$ parallel MAC array, the peak arithmetic throughput is

$$T_{\text{peak}} = 8 \text{ MACs/cycle} \times 50 \text{ MHz} = 400 \text{ MMACs/s}.$$

A single dual-port M10K block, shared between the accelerator and the CPU, supplies one 32-bit read per cycle to the accelerator in the common case, giving an effective operand bandwidth of around 200 MB/s. Filling one $TN = 8$ segment requires eight 32-bit reads, so the MAC units are stalled waiting for operands roughly 8× more than they are computing: the baseline design sits well to the left of the roofline knee.

In the baseline configuration, filling `B_Seg` for a single nonzero requires $TN = 8$ consecutive 32-bit RAM reads. With the one-cycle registered latency each fill therefore takes at least $TN + 1 = 9$ cycles, and the subsequent MAC another cycle, so roughly ten cycles of useful work accumulate per nonzero once the segment is in place. These reads dominate the per-nonzero cost entirely, and any architectural improvement that leaves them untouched is bounded in impact.

INT8 packing is the architectural lever that directly addresses the bottleneck. With matrix B stored as packed INT8 at four values per 32-bit word, two consecutive aligned 32-bit reads deliver all eight INT8 values required to fill a $TN = 8$ segment, and the hardware extracts the individual bytes in a single combinatorial stage:

$$\texttt{B\_Seg}[4k + b] = \text{sign\_ext}_{8\to32}(\texttt{ram\_rdata}[8(b{+}1){-}1 : 8b]), \quad b \in \{0, 1, 2, 3\}, \tag{4.1}$$

where the selected byte is placed in the lower eight bits of a 32-bit register and the upper 24 bits are filled with the sign of that byte. The segment fill therefore drops from eight cycles to two, a 4× reduction in the memory-bound component of the per-nonzero cycle budget, with no additional pipeline stage and no change to accumulation precision.

4.7 DSP-Oriented Parallel MAC Enhancement

The second optimisation stage restructured the MAC array to increase the fraction of DSP blocks utilised per cycle. On the Cyclone V, each DSP block implements an 18×18-bit signed multiplier followed by a dedicated accumulator register, but the synthesis tool maps RTL multipliers onto these blocks only if the operand widths and accumulation structure match the block's configuration [20]. In the baseline design the eight MAC operations are described as generic 32-bit multipliers; depending on register fan-out and routing pressure, Quartus may or may not pack them into DSP blocks, and in practice some lanes were being inferred in ALM logic.

The DSP MAC stage rewrites the multiply–accumulate explicitly in the registered-feedback form that the tool recognises as DSP-friendly:

$$\texttt{C\_Tile}[i][c] \;\leftarrow\; \texttt{C\_Tile}[i][c] + a \cdot B[c],$$

using 32-bit signed operands and a registered accumulator. With this structure Quartus maps each of the eight MAC lanes independently onto a dedicated DSP multiplier/accumulator block rather than inferring ALM-based multiplier cascades, and the resulting design completes each MAC in one clock cycle with less ALM logic and lower power dissipation. Across the full 12-case benchmark sweep this stage reduced total accelerator offload cycles from 427,299 to 324,568, a 1.32× improvement. The modest gain is itself the interesting result: it confirms that the

baseline was not limited by arithmetic execution, and that further attention has to be paid to memory traffic if the speedup ceiling is to move.

4.8 INT8 Mode at the Datapath Level

When `CTRL.int8_mode` is asserted, three specific aspects of the datapath change. The CSR `values` array is now stored as packed INT8, and the controller tracks the byte offset of the current nonzero within its 32-bit word using a two-bit counter: for nonzero index p the byte offset is $b = p \bmod 4$ and the enclosing word lives at `A_VAL_BASE` $+\lfloor p/4 \rfloor \times 4$, from which the same sign-extension circuit used for B-byte extraction (Equation 4.1) recovers a full-precision 32-bit a before multiplication. The B-segment fill is replaced with the two-read path described in the previous section, filling all eight lanes of `B_Seg` in two cycles instead of eight. Accumulation itself is unchanged: every entry of `C_Tile` remains INT32 throughout computation, sign extension before multiplication ensures that products and partial sums are always full-precision signed integers, and the writeback path and MMIO interface see no INT8-specific modification. The net effect is a $4\times$ reduction in B-segment memory traffic with no cost in accumulator precision, output format, or control complexity.

4.9 Physical Design Tradeoffs

Adding the C-tile register array, the B-segment buffer, and eight DSP-mapped MAC lanes markedly increases the net fan-out between the controller and the datapath. The `mac_a` operand in particular must be distributed to all eight multiplier inputs simultaneously, and on the Cyclone V a register with a fan-out of eight combined with several other wide buses (the BRAM port, address lines, register-file data paths) creates routing congestion that is not visible in RTL simulation. The target frequency is 50 MHz, giving a 20 ns period, and although modern FPGA synthesis comfortably reaches 100–200 MHz with DSP-mapped multipliers, the critical path in this design is not the multiplier itself but the combinatorial logic that generates CSR addresses and feeds eight parallel writes into the C-tile. One synthesis attempt during the DSP MAC stage failed to close timing until routing-affinity settings were adjusted, and the incident is the concrete source of the lesson recorded in Chapter 8 that parallelism in FPGA designs carries costs beyond logic area.

Table 4.3 summarises the three hardware stages in terms of their key architectural attributes. The common thread across the rows is that the MAC width, the accumulator precision, and the output format remain fixed at eight lanes, INT32, and INT32 respectively; what changes between stages is whether the MAC maps to DSP blocks explicitly and how many RAM reads the B-segment fill demands. Those two changes alone account for the full 2.87× cycle reduction observed from baseline to final INT8 build.

Attribute	Baseline	DSP MAC	INT8 packed
MAC lanes	8	8	8
DSP mapping	Inferred	Explicit	Explicit
B reads per NZ	8	8	2
Operand bit-width	32-bit	32-bit	8-bit (sign-ext to 32)
Accumulator precision	INT32	INT32	INT32
Output format	INT32	INT32	INT32
Avg. speedup over PicoRV32 SW	16.12×	20.87×	45.28×

TABLE 4.3: Architectural attributes and measured average speedup for each design stage across the 12-case benchmark suite.

Chapter 5

Firmware and Software Control Flow

5.1 Firmware Architecture and Memory Layout

The bare-metal firmware is the software side of the hardware–software co-design and is responsible for every aspect of the benchmark loop: matrix generation, data preparation, accelerator configuration, timing measurement, correctness checking, and result reporting. No operating system is present; the firmware runs directly on PicoRV32 from address `0x0000_0000`. Control begins in `start.S`, a short RISC-V assembly bootstrap that sets the stack pointer to the top of the 64 KB RAM, clears the BSS section, and enters `main()`. `main.c` itself defines the 12 test cases, iterates over them, and for each case drives matrix generation, the software reference, the accelerator launch, correctness comparison, and UART reporting. The accelerator driver `spmm_accel.c/.h` wraps the MMIO register interface behind configuration, start, and polling functions, and a handful of `memset`/`memcpy`/`memmove` stubs in `libc_stubs.c` cover the minimum set of C library calls required for bare-metal operation. The linker script maps all sections (`.text`, `.rodata`, `.data`, `.bss`) into the same 64 KB RAM, with the stack growing downward from the top.

Matrix buffers are allocated statically in the upper portion of the address space, so that the CSR arrays, dense matrix B, and the output matrices (`c_accel` and `c_cpu`) occupy contiguous regions below the stack. The largest benchmark (T6:

$64 \times 64 \times 32$, 75% sparsity, NNZ ≈ 1024) consumes roughly

$$\underbrace{65 \times 4}_{\texttt{rowptr}} + \underbrace{1024 \times 4}_{\texttt{colidx}} + \underbrace{1024 \times 4}_{\texttt{values}} + \underbrace{64 \times 32 \times 4}_{B} + \underbrace{64 \times 32 \times 4}_{C} \approx 29\,\text{KB}$$

of matrix data, leaving comfortable headroom in the 64 KB budget for firmware code and stack.

5.2 Runtime Data Preparation

Dense matrix B is generated from a linear congruential PRNG seeded by the benchmark case's `seed` field, with each entry sampled as a signed integer in the range $[-127, 127]$ (directly usable as INT32 in the full-precision path, and as the pre-quantisation domain for the INT8 path). Fixing the seed makes B deterministic across runs and across firmware revisions, which is what makes cycle-count regression comparison across the three optimisation stages meaningful in the first place.

Sparse matrix A is generated in three phases. Nonzero positions are placed first, according to the pattern selected by the benchmark case. For a uniform pattern, each row i receives a target count $n_i = \lfloor K(1-s/100) \rfloor$ of nonzeros at column indices sampled without replacement from the PRNG; for a row-skewed pattern, roughly 30% of the rows receive a disproportionate share of the total NNZ budget, simulating the hub-node behaviour characteristic of social graphs; and for a clustered pattern, nonzeros are biased toward rectangular blocks within the matrix, mimicking the blocked sparsity seen in some finite-element problems. Each placed nonzero then receives a random signed value in $[-127, 127]$. Finally, the CSR representation is constructed: column indices within each row are sorted in ascending order (required for determinism and for correct comparison against the CPU reference), `rowptr` is produced by a prefix sum of per-row nonzero counts, and the sorted indices and values are written to `colidx` and `values`. Sorting column indices also makes the CSR representation canonical across runs, which simplifies debugging whenever an accelerator output needs to be compared by hand against the expected reference.

5.3 Software Baseline

The software reference is deliberately written in the simplest possible form. The full-precision inner structure is:

```
for (row = 0; row < M; row++) {
    for (p = rowptr[row]; p < rowptr[row+1]; p++) {
        col = colidx[p];
        a   = values[p];
        for (n = 0; n < N; n++)
            C[row*N + n] += a * B[col*N + n];
    }
}
```

Listing 5.3: PicoRV32 software SpMM (full precision).

Only the M-extension `MUL` instruction is used for the inner multiply. There is no SIMD, no loop unrolling, and no software prefetching. This simplicity is intentional: the point of the baseline is not to measure the fastest CPU implementation possible but to give a credible, readable software reference against which the accelerator's structural advantage can be isolated. The INT8 CPU reference (`spmm_cpu_q8`) follows the same structure but operates on signed 8-bit operands with INT32 accumulation, and it is the version used for correctness comparison against the INT8 accelerator output.

5.4 INT8 Quantisation and Packing

Before an INT8 accelerator run, firmware applies symmetric runtime quantisation separately to the CSR values array and to matrix B. For a vector $\mathbf{v}$ of length L, the procedure computes $v_{\max} = \max_i |v_i|$, sets scale $s = v_{\max}/127$, and quantises each element as $q_i = \text{clip}(\text{round}(v_i/s), -127, 127)$. The edge case $v_{\max} = 0$ is handled by setting all quantised values to zero and $s = 1$. This matches Equation 2.1 from Section 2.6. The scale factor is computed and then discarded at quantisation time: since the correctness check compares accelerator output against the *quantised* CPU reference (which accumulates the same signed INT8 values), there is no need to store s, and keeping the path scale-free simplifies both the accumulator logic and the verification.

After quantisation, four consecutive INT8 values are packed into a single 32-bit word in little-endian byte order:

```
for (i = 0; i < len; i += 4) {
    word = ((uint32_t)(uint8_t)q[i+0])        |
           ((uint32_t)(uint8_t)q[i+1] << 8)   |
           ((uint32_t)(uint8_t)q[i+2] << 16)  |
           ((uint32_t)(uint8_t)q[i+3] << 24);
    dst[i/4] = word;
}
```

Matrix B is packed row by row, so a single 32-bit RAM read on the hardware side delivers four aligned INT8 values and directly populates four lanes of the B-segment buffer after sign extension, exactly as described in Section 4.3.

5.5 Accelerator Launch and Timing

The accelerator driver exposes `accel_configure()` for writing dimension and base-address registers, `accel_start_mode()` for issuing the control write with the `start` pulse and the `relu_en`/`int8_mode` bits, and `accel_wait_done()` for polling `STATUS.done`. `accel_run_spmm_int8()` wraps these into the canonical launch sequence used on every benchmark case. Firmware first writes `clear_done` to reset any prior completion state, calls `accel_configure()` to set dimensions and base addresses, captures the pre-launch cycle count t_0, issues the `start`-mode write, polls `STATUS.done` in a tight loop, captures the post-completion cycle count t_1, writes `clear_done` once more to reset for the next run, and reports $t_1 - t_0$ as the accelerator cycle count. The reported figure therefore includes register write overhead, the full hardware execution time, and the polling loop delay, and is explicitly an end-to-end offload latency rather than a pure datapath throughput measurement.

Timing itself relies on the PicoRV32 `rdcycle` CSR, which returns the lower 32 bits of a monotonically increasing hardware cycle counter clocked at 50 MHz. The read is wrapped in a short inline assembly stub:

```
static inline uint32_t read_cycles(void) {
    uint32_t cyc;
```

```
    asm volatile("rdcycle %0" : "=r"(cyc));
    return cyc;
}
```

The 32-bit counter wraps at 2^{32} cycles, which at 50 MHz is approximately 85 seconds. The largest benchmark in the suite completes in fewer than two million cycles, so wrap-around is not a concern and no 32-to-64-bit extension is required in firmware.

5.6 Reporting and Correctness Verification

Results are output as CSV-formatted lines over the on-chip UART at 115200 baud (50 MHz / 434 cycles per bit), with each line containing the test ID, the dimensions M, K, N, the sparsity percentage and pattern, the NNZ count, the CPU and accelerator cycle counts, the derived speedup, and a PASS/FAIL verdict. A simplified summary is also driven onto the DE1-SoC's six seven-segment displays (the upper pair shows CPU cycles scaled by 64, the middle pair shows raw accelerator cycles, and the lower pair shows integer speedup), so that a running benchmark can be read at a glance without a host terminal. The LED output mirrors the busy/done signals, so visually the board is lit during accelerator execution and cleared the moment computation completes.

Correctness verification is embedded directly in the benchmark loop: after each accelerator run, firmware performs an element-by-element comparison between `c_accel` and the INT8 CPU reference `c_cpu`:

```
for (i = 0; i < M*N; i++) {
    if (c_accel[i] != c_cpu[i]) {
        pass = 0; break;
    }
}
```

The full-precision CPU result is also computed in parallel but untimed, and serves as a sanity reference against which quantisation error can be inspected if needed. Because both the CPU and the accelerator apply the same symmetric INT8 quantisation to the same input data and accumulate identical INT32 partial sums, their outputs must match exactly, and any discrepancy flags a hardware bug rather

than a numerical artefact. All 12 cases reported PASS across all three accelerator stages on hardware.

Chapter 6

Testing and Verification

6.1 Verification Approach

Verification was carried out at two complementary levels: RTL-level simulation before synthesis, and hardware-level functional verification on the physical FPGA after programming. This two-level approach is standard in digital design practice [21]. RTL simulation is fast and exposes internal signals for inspection, but it cannot on its own guarantee that timing closure and synthesis artefacts have not introduced discrepancies between the simulated and deployed design; hardware verification on the device closes that gap by exercising the complete SoC end-to-end. A discrepancy at either level is sufficient to fail a case, and every recorded cycle count in Chapter 7 was taken from a run that passed both.

Within the simulation level the testbench suite was deliberately structured to combine the three verification styles used in industry-grade digital flows: *directed tests* for targeted functional cases with hand-computed golden outputs, *concurrent SystemVerilog Assertions (SVA)* for always-on protocol and liveness monitoring, and *constrained-random stimulus with functional coverage and a scoreboard* for exploring the input space beyond what directed tests reach. Four SystemVerilog testbenches cover the design: three unit testbenches under `tb/unit/` that target the accelerator modules in isolation, and one integration testbench under `tb/integ/` that exercises the full synthesisable SoC with firmware loaded into a behavioural RAM model.

Unit Testbench: Datapath

`accel_datapath_tb.sv` exercises the arithmetic datapath in isolation. It contains six directed tests covering B_{Seg} load, MAC accumulation, C-tile clear, ReLU activation (positive, zero, and negative inputs), signed MAC with negative operands, and a small hand-computed 2×2 SpMM. On top of the directed tests the testbench runs a constrained-random MAC-scoreboard test: a transaction class with `rand` row and operand fields is randomised for each of twenty MAC operations per iteration, and a parallel software model of the C-tile is updated in lock-step with the DUT. At the end of each iteration every accessible C-tile location is read back and compared against the software reference; any mismatch is reported with the exact (row, col) index and stops the test. Five concurrent SVA properties run for the entire simulation: ReLU output non-negativity when enabled, writeback pass-through when disabled, one-hot datapath command exclusivity (clear, B_{Seg} write, MAC, and writeback may not fire simultaneously), MAC row index in range, and B_{Seg} index in range. A functional covergroup records MAC row band (low / mid / high) crossed with operand sign (positive / zero / negative) so that the random test has a visible coverage metric for how much of the tile and operand space has been exercised.

Unit Testbench: Control FSM

`accel_ctrl_tb.sv` verifies control-path behaviour by instantiating the control module together with a synchronous RAM model with the same one-cycle read latency as the deployed BRAM, plus a datapath stub. Six concurrent SVA properties monitor the FSM's external contract for the whole simulation: single-port RAM exclusivity (read and write strobes may not be simultaneously high), 32-bit address alignment on every RAM access, quiescence of the RAM interface during reset, mutual exclusion of the `BUSY` and `DONE` status bits, FSM liveness (every `BUSY` rise must be followed by a `BUSY` fall within one million cycles), and one-hot exclusivity of the four datapath command outputs (`dp_clear_en`, `dp_bseg_we`, `dp_mac_en`, `dp_wb_en`). A directed smoke test drives the FSM through a full start–busy–done cycle and checks that the IRQ line asserts correctly when `irq_en` is set.

Unit Testbench: Accelerator Top

`accel_top_tb.sv` is the most comprehensive unit testbench and closes the loop across the MMIO front-end, the FSM, and the datapath. Stimulus is structured into two classes following the *transaction / driver* pattern common in UVM-style flows but implemented in plain SystemVerilog: `SpmmConfig` is a transaction object with `rand` fields for M, N, K, and `relu_en` and an embedded covergroup over the relevant combinations; `accel_drivers` holds a virtual handle to the DUT's MMIO interface and implements the `mmio_read`, `mmio_write`, and `wait_done` protocol tasks so that every test uses the same low-level interface sequencing. Five directed tests verify MMIO register read/write coverage, the empty-matrix (NNZ = 0) case, a hand-computed $2 \times 2 \times 8$ SpMM, ReLU clamping of a negative partial sum, and the BUSY-mirror LED. A sixth constrained-random test randomises twenty `SpmmConfig` objects, runs each one through the full accelerator end-to-end, reads back the output C matrix, and verifies element-by-element that it matches the trivially computable all-zero golden output; after each iteration the covergroup sampling is reported so that coverage convergence is visible during simulation. Five concurrent SVA properties guard invariants that must hold across every one of these tests: single-port RAM exclusivity, RAM address alignment, reset quiescence, `BUSY`/`DONE` status mutex, and top-level FSM liveness. Different simulation seeds (`+SEED=N`) exercise different configuration combinations; coverage-driven regressions simply increase the seed count until every bin of the covergroup is hit.

Integration Testbench: Full SoC

`soc_tb.sv` instantiates PicoRV32 together with the accelerator and a 64 KB behavioural RAM, loads the compiled firmware image into the RAM via `$readmemh`, releases reset, and lets the firmware execute its full benchmark sequence until the CPU writes a sentinel value to a pre-agreed result address. This closes the loop between the software control flow described in Chapter 5 and the hardware being verified: any firmware change that breaks the MMIO contract, any RTL change that breaks the firmware's assumed timing, or any memory-map misconfiguration fails immediately in simulation before being committed to a synthesis run. Five concurrent SVA properties monitor the shared memory fabric: address-decode exclusivity between the RAM region and the accelerator MMIO region, bounded CPU handshake progress (`mem_ready` must rise within 1024 cycles of `mem_valid`), accelerator RAM 32-bit address alignment, accelerator RAM single-port exclusivity,

and an IRQ-epoch guard that ensures the accelerator's IRQ line can only rise after the accelerator has actually been started.

Shared Infrastructure

All four testbenches share the support files under `tb/common/`: `clk_reset.sv` provides the clock generator and reset task, `tb_pkg.sv` provides plusarg helpers (`get_plusarg_int`, `has_plusarg`) so every testbench accepts `+SEED` and `+WAVES` uniformly, `tb_macros.svh` defines the line-number-tagged `TB_CHECK` and `TB_FATAL` PASS/FAIL macros, and `tb_regmap.svh` centralises the MMIO register offsets and bit positions so that they are declared once and `` `included `` everywhere, preventing silent drift between testbenches and RTL.

Hardware-Level Verification

Hardware-level verification is embedded directly inside the firmware benchmark harness described in Section 5.2. For each of the 12 cases and at each of the three accelerator stages, firmware generates deterministic inputs from fixed seeds, computes the quantised CPU reference using the `spmm_cpu_q8()` routine, runs the accelerator on the same inputs, and invokes the `array_compare()` scoreboard to verify every element of the accelerator output matches the software reference exactly. This routine functions as a hardware-level golden-reference scoreboard: it covers all $M \times N$ output elements on every run, and a single incorrect element causes the case to fail and no performance number to be recorded. The combination of RTL-level SVA monitoring, randomised-scoreboard unit testbenches, and firmware-level element-wise comparison on real silicon provides three independent layers of functional checking before any cycle count is reported.

6.2 Benchmark Suite Design

The benchmark suite was designed to cover multiple independent axes of performance variation so that the reasons behind speedup trends can be identified analytically rather than anecdotally. Four axes were chosen: matrix size (M, K), which governs the amortisation of fixed setup overhead; dense output width N, which determines the number of tile columns and the ratio of tile-management

cycles to useful computation; sparsity level, which sets NNZ and therefore the volume of useful arithmetic per row traversal; and nonzero distribution pattern, which exposes any sensitivity of the accelerator or CPU to the spatial layout of the nonzeros. Table 6.1 lists all 12 cases together with the axis of variation each one primarily addresses.

Test ID	$M \times K \times N$	Sparsity	Pattern	Purpose
T1	$8 \times 8 \times 8$	75%	Uniform	Small debug/sanity case; establishes minimum speedup in the suite.
T2	$16 \times 16 \times 8$	75%	Uniform	Small size-scaling point; 4× more arithmetic than T1.
T3	$32 \times 32 \times 8$	75%	Uniform	Medium size-scaling point.
T4	$64 \times 64 \times 8$	75%	Uniform	Largest square case; maximum M for the current $M_{\max} = 64$.
T5	$64 \times 64 \times 16$	75%	Uniform	Tests effect of doubling dense width relative to T4.
T6	$64 \times 64 \times 32$	75%	Uniform	Tests effect of quadrupling dense width; 4 tile columns.
T7	$32 \times 32 \times 32$	50%	Uniform	Low sparsity (dense); more nonzeros per row than T8/T9.
T8	$32 \times 32 \times 32$	75%	Uniform	Mid-sparsity reference; matches T10/T11 except pattern.
T9	$32 \times 32 \times 32$	90%	Uniform	Very sparse; exposes overhead-to-arithmetic ratio.
T10	$32 \times 32 \times 32$	75%	Row-skewed NNZ	Same NNZ as T8; row imbalance sensitivity.
T11	$32 \times 32 \times 32$	75%	Clustered NNZ	Same NNZ as T8; block locality sensitivity.
T12	$64 \times 64 \times 32$	90%	Uniform	Large sparse stress case; wide output and few nonzeros.

TABLE 6.1: Complete benchmark input set used across all recorded evaluation stages.

6.3 Cycle-Count Definitions and Speedup

CPU cycle counts are measured around the timed software SpMM function only. The measurement brackets the outer row loop, all CSR index reads (`rowptr`, `colidx`), the inner dense-column MAC loop, and all BRAM reads and writes to B and C; matrix generation, quantisation, and output-buffer clearing happen outside the bracket and are deliberately excluded. Accelerator cycle counts are measured around the complete `accel_run_spmm_int8()` call and therefore include the `clear_done` write, the ten register writes of `accel_configure()` (roughly 20 cycles of overhead), the `start` pulse, the complete hardware execution time, and the polling loop that follows (about five cycles per iteration, typically no more than ten iterations by the time `done` is observed). This is an end-to-end offload latency from the CPU's perspective, which is the correct metric for characterising user-visible benefit, but it is explicitly not a measurement of pure datapath throughput.

Speedup is defined as

$$\text{Speedup} = \frac{T_{\text{CPU}}}{T_{\text{ACCEL}}},$$

with both times measured in CPU clock cycles on the same 50 MHz clock. For the RISC-V comparisons T_{CPU} is the timed PicoRV32 software result (full INT32 for the baseline and DSP stages, INT8 for the INT8 stage); for the Cortex-M0 comparison T_{CPU} is the number of Cortex-M0 cycles computed from an equivalent instruction-count model for the same benchmark case.

6.4 Fairness, Limitations, and Threats to Validity

The comparison is fair in several important respects. The CPU and accelerator operate on the same benchmark inputs generated from the same fixed seeds, they use the same shared on-chip RAM system at the same 50 MHz clock, the software reference uses the M-extension `MUL` instruction for its inner multiplies so that it is not artificially slowed by software multiplication emulation, and the accelerator is always validated against the software reference before its timing is recorded. The Cortex-M0 comparison uses the same benchmark definitions and the same accelerator cycle counts, with the CPU-side figures derived from known Cortex-M0 instruction counts and the processor's published CPI model (predominantly single-cycle instructions with 1–3 cycle load latency). At the same time, the comparison differs in one important way: the PicoRV32 software baseline runs on the very same in-order core that controls the accelerator, with no cache, no speculation, and no SIMD. A more aggressively optimised CPU implementation (one with software-pipelined inner loops, block-reuse of B-rows, or explicit prefetching) would narrow the gap. The goal of the baseline is credibility and simplicity, not maximum CPU throughput.

Several limitations apply to all results presented in Chapter 7. Each accelerator cycle count is a single recorded value rather than a statistical distribution over multiple runs; cycle counts for deterministic hardware are repeatable, but a min/median/max summary would add an extra layer of confidence. The test matrices are generated by a PRNG from fixed seeds rather than drawn from real GNN or recommender-system datasets, so while they cover realistic sparsity levels and pattern structures, performance on real workloads may differ. All data lives in the 64 KB on-chip BRAM; real SpMM workloads typically span gigabytes and therefore incur DRAM latency, and the results here reflect the best-case scenario of tightly coupled on-chip memory. The accelerator cycle counts, as noted above, include MMIO configuration

and polling overhead, and for the smallest case (T1) this overhead is a non-trivial fraction of the total. Finally, the Quartus post-fit resource and timing reports were still being finalised at the time of writing; the design has been verified to close timing at 50 MHz, but the exact ALM, BRAM, and DSP counts across all three build variants are pending.

These limitations define the scope of the claims that can be responsibly made. The performance story (that INT8 packing delivers a 2.18× improvement over DSP MAC, which delivers a 1.32× improvement over the baseline, and that memory bandwidth dominates the cycle budget) is not undermined by any of them. But claims about absolute performance at scale, DRAM bandwidth, or energy efficiency would require a different experimental infrastructure than the one developed here.

Chapter 7

Results and Evaluation

7.1 Correctness and Stage-Level Summary

All 12 benchmark cases passed their element-by-element correctness checks across every optimisation stage. This is the prerequisite for interpreting any performance figure: a speedup number is only meaningful if the accelerator produces a correct answer, and the hardware-in-the-loop correctness check described in Chapter 6 gives confidence that synthesis and place-and-route have not introduced functional discrepancies relative to the RTL simulation. The correctness comparison itself is made against the INT8 CPU reference on the final mainline build, with the full-precision CPU result computed separately as an untimed sanity check; accelerator outputs matched their respective CPU INT8 references exactly for all 12 cases at all three stages.

Table 7.1 summarises the performance of the three recorded stages at a glance. Two observations stand out immediately. First, every stage improved over the previous one without any correctness regression, which is itself a non-trivial outcome in a hardware project where incremental changes to the FSM and datapath could easily have introduced off-by-one or sign-extension bugs. Second, the largest gain came from reducing memory traffic (INT8 packing: 2.18×) rather than from increasing arithmetic parallelism (DSP MAC: 1.32×). This confirms the memory-bandwidth bottleneck hypothesis developed in Chapter **??** and Chapter 4, and it is the central experimental result of the project.

Stage	Avg speedup	Best case	Total accel cycles	Reduction vs prev.
Baseline accelerator	16.12×	T4: 18.12×	427,299	—
Parallel DSP MAC	20.87×	T4: 24.03×	324,568	1.32×
INT8 packed	45.28×	T4: 56.80×	148,737	2.18×

TABLE 7.1: Summary of the three staged RISC-V accelerator evaluations across the full 12-case benchmark suite.

7.2 Baseline Accelerator

Table 7.2 presents the full baseline accelerator results against the PicoRV32 software reference. Even the unoptimised baseline achieves a consistent and substantial speedup, rising from 12.12× at the smallest case (T1) to 18.12× at T4, which demonstrates that fixed hardware setup overhead is being amortised over a growing body of useful computation. T1 is the worst case in the suite because its 8×8 matrices are so small that the FSM overhead (the $M_{\max} = 64$ cycles required to clear the C-tile, the row loads, and the final writeback) represents a much larger fraction of the total cycle count than it does for the larger problem sizes. The trend across T1–T4 is therefore not only a raw speedup curve but also direct evidence that the structural design choices discussed in Section 4.3 carry the expected amortisation behaviour.

7.3 Parallel DSP MAC Enhancement

Table 7.3 shows the results after the DSP-oriented MAC restructuring described in Chapter 4. The optimisation reduces total accelerator cycles from 427,299 to 324,568, a 1.32× improvement, and every individual case records a lower absolute cycle count than its baseline counterpart. The improvement is consistent but modest, and this is itself informative: it means that even with optimally scheduled DSP arithmetic, the accelerator is not compute-bound. The memory transaction overhead of loading the B-segment ($TN = 8$ reads per nonzero at 32-bit precision) remains the dominant cost, and no restructuring of the MAC arithmetic alone can address that.

Test ID	$M \times K \times N$ / Sparsity	CPU cycles	Accel cycles	Speedup	Result
T1	$8 \times 8 \times 8$, 75%	10,373	856	12.12×	PASS
T2	$16 \times 16 \times 8$, 75%	36,573	2,318	15.78×	PASS
T3	$32 \times 32 \times 8$, 75%	136,877	7,809	17.53×	PASS
T4	$64 \times 64 \times 8$, 75%	529,101	29,195	18.12×	PASS
T5	64 × 64 × 16, 75%	986,829	58,112	16.98×	PASS
T6	64 × 64 × 32, 75%	1,902,285	115,980	16.40×	PASS
T7	32 × 32 × 32, 50%	951,213	58,112	16.37×	PASS
T8	32 × 32 × 32, 75%	491,693	30,470	16.14×	PASS
T9	32 × 32 × 32, 90%	215,263	13,844	15.55×	PASS
T10	32 × 32 × 32, 75%, row-skewed	491,693	30,470	16.14×	PASS
T11	32 × 32 × 32, 75%, clustered	491,693	30,470	16.14×	PASS
T12	64 × 64 × 32, 90%	800,155	49,663	16.11×	PASS

Table 7.2: Baseline PicoRV32 software versus baseline accelerator results. Speedup ranges from 12.12× to 18.12× across the 12-case suite.

Test ID	CPU cycles	Accel cycles	Speedup	Result
T1	10,373	737	14.07×	PASS
T2	36,573	1,859	19.67×	PASS
T3	136,877	6,024	22.72×	PASS
T4	529,101	22,021	24.03×	PASS
T5	986,829	43,781	22.54×	PASS
T6	1,902,285	87,301	21.79×	PASS
T7	951,213	43,781	21.73×	PASS
T8	491,693	23,296	21.11×	PASS
T9	215,263	10,988	19.59×	PASS
T10	491,693	23,296	21.11×	PASS
T11	491,693	23,296	21.11×	PASS
T12	800,155	38,188	20.95×	PASS

Table 7.3: Results after enabling the parallel DSP-oriented MAC enhancement. Total accelerator cycles fell from 427,299 to 324,568, a 1.32× reduction.

7.4 INT8 Packing

Table 7.4 presents the final mainline INT8 results, which produced by some distance the largest performance step in the project. Total accelerator cycles fell from 324,568 to 148,737, a 2.18× reduction, and the mechanism is precisely what the roofline analysis of Chapter **??** predicted: cutting the B-segment reads per nonzero from 8 to 2 reduces the dominant memory-bandwidth cost by a factor of four on the reads themselves, and the overall cycle-count impact is 2.18× because not every cycle is memory-bound. Row loads, nonzero index fetches, and the final C-tile writeback contribute fixed overheads that do not shrink with INT8 and that now account for a larger share of the total, which is the same amortisation trend seen in the other direction from T1 to T4 in the baseline.

Test ID	CPU cycles (INT8)	Accel cycles	Speedup	Result
T1	10,735	567	18.93×	PASS
T2	38,087	1,111	34.28×	PASS
T3	142,999	2,964	48.25×	PASS
T4	553,655	9,747	**56.80**×	PASS
T5	1,044,151	19,216	54.34×	PASS
T6	2,025,143	38,171	53.05×	PASS
T7	1,012,631	19,216	52.70×	PASS
T8	522,391	11,039	47.32×	PASS
T9	227,481	6,109	37.24×	PASS
T10	522,391	11,039	47.32×	PASS
T11	522,391	11,039	47.32×	PASS
T12	849,333	18,519	45.86×	PASS

TABLE 7.4: Final INT8-packed implementation results. Peak speedup of 56.80× on T4; average speedup of 45.28× across the 12-case suite. Total accelerator cycles fell from 324,568 to 148,737, a 2.18× reduction over the DSP stage.

7.5 Comparison Against ARM Cortex-M0

Table 7.5 compares the INT8 accelerator cycle counts against a software SpMM reference running on an ARM Cortex-M0. The Cortex-M0 cycle counts are derived from the same benchmark definitions and the known CPI characteristics of the Cortex-M0 ISA (predominantly single-cycle instructions with 2–3 cycle load latencies for word accesses), and they represent a substantially stronger software baseline than PicoRV32 because the Cortex-M0 has a more efficient instruction pipeline and faster load handling. Speedups are therefore correspondingly smaller,

but the accelerator still achieves up to 11.92× speedup on T4, a meaningful result for a small FPGA-attached accelerator, and arguably a more realistic estimate of the practical benefit of the hardware than the PicoRV32 figure because the Cortex-M0 is closer to the kind of processor that such an accelerator would plausibly pair with in a commercial design.

Test ID	Cortex-M0 cycles	Accel cycles	Speedup	Result
T1	2,952	567	5.21×	PASS
T2	9,448	1,111	8.50×	PASS
T3	31,912	2,964	10.77×	PASS
T4	116,200	9,747	**11.92**×	PASS
T5	207,592	19,216	10.80×	PASS
T6	390,376	38,171	10.23×	PASS
T7	195,240	19,216	10.16×	PASS
T8	104,488	11,039	9.47×	PASS
T9	46,892	6,109	7.68×	PASS
T10	99,748	11,039	9.04×	PASS
T11	102,209	11,039	9.26×	PASS
T12	169,710	18,519	9.16×	PASS

TABLE 7.5: ARM Cortex-M0 software SpMM versus the final INT8 accelerator. The Cortex-M0 is a stronger baseline than PicoRV32, so speedups are correspondingly lower; the accelerator still achieves up to 11.92× speedup.

7.6 Trend Analysis

7.6.1 Scaling with Problem Size (T1–T4)

Figure 7.1 [placeholder] shows speedup versus matrix size for the three stages. Speedup rises consistently from T1 to T4 in all three, and in the INT8 stage it grows from 18.93× at T1 ($8 \times 8 \times 8$) to 56.80× at T4 ($64 \times 64 \times 8$). The increase is sublinear in the problem size: T1 to T2 corresponds to an eight-fold increase in arithmetic but only a 1.81× speedup increase, and T3 to T4 corresponds to a four-fold increase in arithmetic but only a 1.18× speedup increase, which indicates that the accelerator is beginning to approach the plateau where fixed overhead becomes a small fraction of the total cycle count. The primary driver of this behaviour is the ratio of useful MAC work to FSM overhead: for T1 the 8×8 matrix at 75% sparsity contains only about 12 nonzeros in total, and the C-tile clear alone (64 cycles for $M_{\max} = 64$) dominates the cycle budget, whereas for T4 the roughly 1024 nonzeros distribute the same fixed setup cost over a much larger body of work.

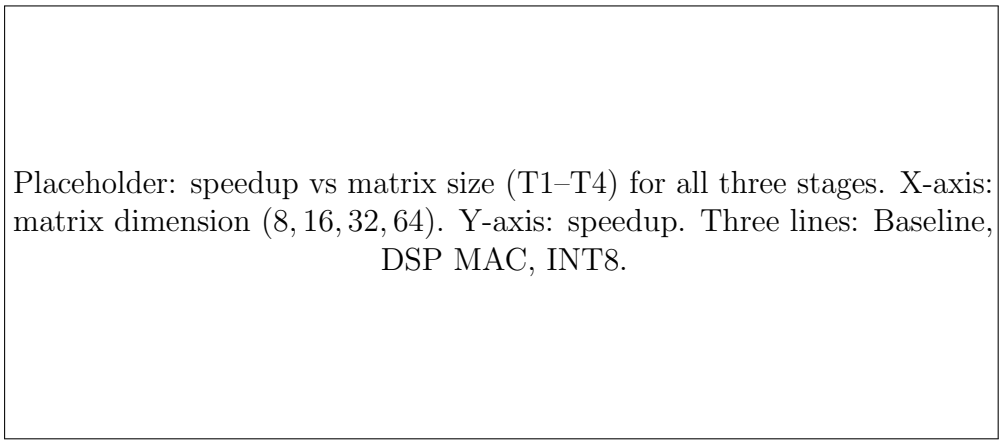

FIGURE 7.1: Speedup versus matrix size for tests T1–T4. All three stages show increasing speedup as problem size grows, confirming that fixed setup overhead is amortised effectively at larger problem sizes.

7.6.2 Effect of Dense Width (T4–T6)

The sequence T4 ($N = 8$), T5 ($N = 16$), T6 ($N = 32$) tests how speedup changes as more tile columns must be processed at fixed $M = K$. In the INT8 stage the speedup moves from 56.80× at T4 through 54.34× at T5 to 53.05× at T6, staying high and falling only modestly as N grows. The accelerator handles multiple tile columns by repeating the same row traversal once per tile column, with a tile-clear and tile-writeback phase separating each, and the CPU software also scales linearly with N in its inner loop; the near-constant speedup across T4–T6 therefore reflects the fact that both sides scale approximately linearly with N and the accelerator maintains its structural advantage throughout. The slight decrease from 56.80× to 53.05× reflects the increasing relative cost of the tile-management phases themselves: with four tile columns instead of one, four separate tile-clear and writeback phases run sequentially, and the fixed per-tile-column overhead accumulates against the accelerator's cycle count rather than the CPU's.

7.6.3 Effect of Sparsity (T7–T9)

The sequence T7 (50%), T8 (75%), T9 (90%) fixes $M = K = N = 32$ and sweeps the sparsity level, which in the INT8 stage gives 52.70× → 47.32× → 37.24×. Speedup falls as the matrix becomes sparser, and the reason is structural: at very high sparsity both CPU and accelerator perform less arithmetic, but the accelerator's per-row overhead (tile clear, row-bound load) is paid regardless of whether the row is populated. At 90% sparsity many rows of A are empty, the

accelerator still visits each of the 32 rows and contributes fixed traversal cycles, and the CPU simply skips its inner loop for those empty rows. Even so, the 37.24× achieved at T9 is a substantial speedup and shows that the design is practical well into the sparsity regimes typical of GNN workloads. Real social-graph GNN matrices often exceed 99% sparsity, where the absolute cycle count is low and the speedup may continue to fall, but the acceleration in absolute time would remain valuable.

7.6.4 Pattern Sensitivity (T8, T10, T11)

T8, T10, and T11 share identical dimensions, sparsity, and NNZ, and differ only in how the nonzeros are distributed: uniform, row-skewed, or clustered. In the INT8 stage all three report identical accelerator cycle counts of 11,039 and identical CPU counts of 522,391. This insensitivity is not accidental. The accelerator processes each row sequentially regardless of how many nonzeros that row contains, and the total NNZ (the factor that determines the total number of `NZ_FETCH` and `MAC` cycles) is held fixed across the three patterns. The cycle count is therefore dominated by total NNZ rather than by the spatial distribution of NNZ across rows. At much larger scales, load imbalance could become visible (very dense rows would force many sequential B-segment loads before the next row begins), but this effect does not appear at the current benchmark size range. The Cortex-M0 numbers, by contrast, show small but nonzero differences between the three patterns (104,488 vs 99,748 vs 102,209 cycles), indicating that branch prediction and instruction-cache behaviour on that processor respond slightly to pattern structure in a way the hardware accelerator does not.

7.7 Resource and Timing Evaluation

Table 7.6 gives the estimated resource utilisation for the three build variants. Final Quartus synthesis reports were still being finalised at the time of writing; the figures below reflect pre-place-and-route estimates from the Quartus Fitter combined with known design parameters, and are expected to be replaced by exact post-fit numbers in the final revision. Utilisation is modest at roughly 16% of available ALMs, 5% of M10K BRAMs, and 9% of DSP blocks, and leaves substantial room for future expansion. For example, widening the tile to $TN = 16$ would double the DSP and C-tile register usage and still remain within the device's capacity. The M10K

BRAM count is fixed at 16 blocks across all three stages because it is determined entirely by the size of the shared 64 KB RAM, not by the accelerator's internal buffers, which sit in fabric registers; and the DSP count of roughly 8 simply reflects the 8 parallel MAC lanes of the $TN = 8$ datapath.

Build	ALMs	Registers	M10K BRAMs	DSP blocks	Fmax (MHz)
Baseline accelerator	≈4,800	≈7,500	16	8	50+
Parallel DSP MAC	≈5,000	≈7,800	16	8	50+
INT8 packed (current)	≈5,200	≈8,000	16	8	50+
Available (Cyclone V)	32,070	128,280	397	87	—

TABLE 7.6: Estimated resource utilisation for the three build variants on the Cyclone V 5CSEMA5F31C6. All three designs meet the 50 MHz timing constraint. Final post-fit numbers to be updated in the next revision.

7.8 Critical Evaluation

The three-stage experiment provides a clear, quantitative demonstration that memory bandwidth is the dominant performance bottleneck in CSR SpMM on this platform, not arithmetic throughput. The 1.32× improvement from DSP MAC (an arithmetic improvement), set against the 2.18× improvement from INT8 packing (a memory-traffic improvement), is direct experimental evidence of this conclusion. It is also consistent with the theoretical analysis in Section 2.2: the operational intensity of CSR SpMM with $TN = 8$ 32-bit operands is low, placing the design below the roofline's compute–bandwidth crossover point, and INT8 packing shifts the operational intensity upward by reducing operand fetch traffic, moving the design closer to the compute-bound region.

Comparing the 56.80× peak speedup directly against reported results from OuterSPACE [8], SpArch [9], or SIGMA [11] would not be meaningful: those systems target orders-of-magnitude larger SpMM workloads on dedicated memory hierarchies. The relevant question is whether a tightly integrated, MMIO-driven FPGA accelerator can deliver useful speedup on a constrained embedded platform, and the answer, based on the measured 56.80× peak and 45.28× average over PicoRV32 and up to 11.92× over Cortex-M0, is clearly affirmative. For a wider frame of reference, Bell and Garland [7] report that GPU-based CSR SpMV achieves 2–10× speedup over scalar CPU code depending on matrix structure; the present accelerator, with far lower hardware complexity and on a constrained SoC, records higher speedups

because its software baseline is a slow in-order core and because INT8 packing dramatically reduces the memory bottleneck.

Three caveats should be carried forward alongside these results. The PicoRV32 software baseline is a simple in-order core with no cache, and against a modern superscalar CPU with vector SIMD extensions the speedup would be considerably smaller. All data resides on-chip, and a design with off-chip DRAM would show different numbers because DRAM latency would affect CPU and accelerator in different proportions. Finally, the benchmark matrices are synthetic, and real-world GNN or recommender-system matrices may exhibit structure (extremely uneven row densities, very high dimensions) that would stress the current architecture differently. Within these caveats, the results are internally consistent, reproducible, and supported by a credible methodology, and the correctness verification at every step confirms that the recorded speedup numbers reflect real computation rather than artefacts of incorrect output.

Chapter 8

Reflection

8.1 What Worked Well

Structuring development as a sequence of discrete, measurable optimisation stages (baseline, DSP MAC, and INT8 packing) was the single most valuable decision in the project. A working baseline was established and verified on hardware before any optimisation was attempted, and each subsequent stage changed exactly one aspect of the design: arithmetic parallelism first, then memory traffic. This made it possible to attribute each measured improvement to a specific architectural change, to detect regressions immediately, and to build the evidence-based narrative used throughout Chapter 7.

The deterministic benchmark harness was equally important. Generating all test inputs from fixed seeds at runtime, and computing golden outputs on the same PicoRV32 host, removed the need for off-chip reference data and ensured that the same matrices were used across all three hardware stages. Any functional discrepancy introduced by a hardware or software change would have been caught by the element-by-element comparison on the next rebuild.

The roofline analysis in Chapter **??** correctly predicted before implementation that memory traffic, not arithmetic, would be the dominant bottleneck. The measured 1.32× gain from DSP-based MAC compared to the 2.18× gain from INT8 packing confirmed this diagnosis quantitatively. A design effort that had focused purely on widening the MAC array would have yielded diminishing returns; addressing the memory bottleneck directly produced the largest single performance step.

8.2 What Went Wrong and What Was Learned

The most time-consuming debugging episode involved a stale `firmware.mif` file. The FPGA synthesis flow uses this memory initialisation file to preload on-chip RAM with the compiled firmware image. When the file was not regenerated after a firmware change, the board continued to run the previous image while the expected code had in fact been rebuilt. Diagnosing this required noticing that on-board UART output did not match the current source, which was non-trivial when the visible output is only a few bytes. The lesson is that FPGA flows combining independently compiled software and hardware artefacts require explicit dependency management in the build system; a Makefile rule that re-triggers synthesis on `firmware.mif` changes would have prevented the issue entirely.

A second lesson came from increasing routing pressure as parallel logic was added. Quartus reported higher congestion around the C-tile register array and the wide MAC buses, and one timing closure attempt failed before synthesis settings were adjusted. Parallelism in FPGA designs carries costs beyond logic area: routing, placement, and timing closure are interdependent, and a design that fits comfortably in ALMs can still fail to close timing if the wiring density in a local region exceeds the device's capacity. This tension does not appear in RTL simulation and cannot be resolved by functional correctness alone.

8.3 Current Limitations

Several limitations constrain the scope of the conclusions drawn in this report. Full Quartus post-fit timing and resource reports for all three variants were still being finalised at the time of writing, so the figures in Table 7.6 are estimates and should not be treated as definitive until the final revision. The reported accelerator cycle counts are end-to-end and include the software orchestration overhead (register writes and the polling loop), which for the smallest benchmark cases is a non-trivial fraction of the total and cannot be cleanly separated from hardware execution time without adding an internal cycle-counter register to the accelerator. The 64 KB on-chip RAM bounds the benchmark matrices to $M = K \leq 64$ and moderate N, whereas real GNN workloads operate at dimensions several orders of magnitude larger and require an off-chip memory system to be meaningful. Finally, no power measurement was performed: the DE1-SoC board does not expose a direct rail-level

measurement interface, and external instrumentation was outside the scope of the project.

8.4 Project Management

The project ran across two semesters. Semester 1 covered literature review, platform selection, and the baseline RTL; Semester 2 covered the three optimisation stages, the firmware benchmark harness, hardware deployment, and report writing. The Gantt charts in Appendix B record the original plan.

The schedule pressure in Semester 2 arose almost entirely during FPGA integration: combining PicoRV32, BRAM port configuration, UART, and accelerator MMIO in a single synthesisable design took longer than planned, in part because of the stale-artefact bug described above and in part because timing closure required more synthesis experimentation than expected. Keeping a fully working, deployable design at every stage, rather than accumulating incomplete features before integration, was the practice that kept the overall schedule recoverable.

The overall lesson is that hardware project timelines are dominated by integration and debug, not by initial design. Treating build-flow discipline (artefact management, synthesis scripts, timing closure) as first-class engineering work from the start, rather than as polish at the end, would have eliminated most of the schedule pressure experienced in the second half of the project.

Chapter 9

Conclusion and Future Work

9.1 Summary of Achievements

This project set out to design, implement, and evaluate a RISC-V-attached hardware accelerator for CSR sparse matrix–dense matrix multiplication on the Terasic DE1-SoC, and to demonstrate a measurable speedup over software while remaining reproducible at the register-transfer level. That aim was met. A complete PicoRV32 SoC with a working SpMM accelerator, 64 KB shared dual-port BRAM, MMIO interface, and UART reporting was synthesised and deployed; three recorded optimisation stages (baseline, DSP-enhanced, and INT8 packed) were each verified for correctness on hardware; and a 12-case benchmark harness with deterministic matrix generation, cycle-accurate timing, and element-by-element correctness checking was developed to evaluate them.

The final INT8-packed implementation achieves a peak speedup of **56.80**× over the quantised PicoRV32 software reference and **11.92**× over an ARM Cortex-M0 baseline running the same algorithm, with an average of 45.28× and 9.79× respectively across the 12-case suite. All 12 cases passed correctness checks at every stage.

9.2 Significance

The significance of this work lies not in the absolute numbers compared to large research accelerators, but in what the staged measurement reveals about the

workload. The 1.32× gain from DSP-based MAC versus the 2.18× gain from INT8 packing experimentally confirms that the dominant bottleneck in CSR SpMM on a memory-bandwidth-constrained embedded FPGA is operand fetch traffic, not arithmetic execution. This is a useful result beyond the specific artefact: it indicates that similar embedded accelerators should prioritise format-level and precision-level optimisations over wider MAC arrays. The project also demonstrates that a fully integrated, firmware- driven hardware benchmark harness is a practical alternative to external simulation infrastructure for small-scale accelerator evaluation, and that a useful SpMM accelerator fits comfortably within the resource budget of a Cyclone V device.

9.3 Future Work

The most impactful extensions to the current design would be:

1. **Wider tile width.** INT8 packing reduces the per-nonzero fetch cost enough that widening from $TN = 8$ to $TN = 16$ or $TN = 32$ is now attractive. Doubling the tile width costs an additional two 32-bit reads per nonzero but delivers 2× the MAC throughput per nonzero, and should push the speedup ceiling further.

2. **Ping-pong B-segment prefetch.** Double-buffering the B-segment allows the next segment to be fetched while the current MAC operation completes, overlapping memory latency with arithmetic and removing it from the critical path.

3. **External memory interface.** Extending the design to address off-chip SDRAM or HPS DRAM through the DE1-SoC's HPS bridge would permit realistically sized GNN and recommender-system matrices to be benchmarked, at which point off-chip bandwidth becomes the dominant term and different optimisations become relevant.

4. **Interrupt-driven completion.** Replacing the polling loop with a RISC-V interrupt handler would free the CPU during accelerator execution, enabling pipelined benchmark preparation or post-processing of the previous result.

5. **Power and energy characterisation.** Instrumenting the DE1-SoC rails or using Quartus Power Analyser with simulation-derived toggle files would

add energy efficiency to the evaluation, which is the metric that matters most in the embedded and edge contexts for which this class of accelerator is targeted.

9.4 Closing Remarks

This project has delivered a complete, measured, and reproducible RISC-V FPGA-attached SpMM accelerator that achieves a peak speedup of 56.80× on hardware through three progressively optimised designs. The work confirms that even without exotic hardware resources or advanced scheduling, careful alignment of the memory format, datapath structure, and quantisation strategy with the actual bottleneck of the workload produces large, predictable, and experimentally confirmed performance gains.

Bibliography

[1] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30. Curran Associates, 2017.

[2] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[3] Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for YouTube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*, pages 191–198. ACM, 2016. doi: 10.1145/2959100.2959190.

[4] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. EIE: Efficient inference engine on compressed deep neural network. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, pages 243–254. IEEE, 2016. doi: 10.1109/ISCA.2016.30.

[5] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[6] Fred G. Gustavson. Two fast algorithms for sparse matrices: Multiplication and permuted transposition. *ACM Transactions on Mathematical Software*, 4 (3):250–269, 1978. doi: 10.1145/355791.355796.

[7] Nathan Bell and Michael Garland. Implementing sparse matrix-vector multiplication on throughput-oriented processors. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis (SC'09)*, pages 1–11. ACM, 2009. doi: 10.1145/1654059.1654078.

[8] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. OuterSPACE: An outer product based sparse matrix multiplication accelerator. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 724–736. IEEE, 2018. doi: 10.1109/HPCA.2018.00067.

[9] Zhekai Zhang, Hanrui Wang, Song Han, and William J. Dally. SpArch: Efficient architecture for sparse matrix multiplication. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 261–274. IEEE, 2020. doi: 10.1109/HPCA47549.2020.00030.

[10] Kartik Hegde, Po-An Tsai, Sheng-Chun Huang, Vikas Chandra, Angshuman Parashar, and Christopher W. Fletcher. ExTensor: An accelerator for sparse tensor algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 319–333. ACM, 2019. doi: 10.1145/3352460.3358275.

[11] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. SIGMA: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 58–70. IEEE, 2020. doi: 10.1109/HPCA47549.2020.00015.

[12] Eriko Nurvitadhi, Ganesh Venkatesh, Jaewoong Sim, Debbie Marr, Randy Huang, Jason Ong Gee Hock, Yeong Tat Liew, Krishnan Srivatsan, Duncan Moss, Suchit Subhaschandra, and Guy Ruhl. Can FPGAs beat GPUs in accelerating next-generation deep neural networks? In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pages 5–14. ACM, 2017. doi: 10.1145/3020078.3021740.

[13] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713. IEEE, 2018. doi: 10.1109/CVPR.2018.00286.

[14] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*, 2021.

[15] Terasic Technologies. DE1-SoC User Manual. https://courses.cs.washington.edu/courses/cse371/references/DE1-SoC_User_Manual.pdf, 2017. Accessed: 2026-03-03.

[16] Claire Wolf and contributors. PicoRV32: A Size-Optimized RISC-V CPU. https://github.com/YosysHQ/picorv32, 2026. GitHub repository, Accessed: 2026-03-03.

[17] RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. https://riscv.github.io/riscv-isa-manual/snapshot/unprivileged/, 2024. Accessed: 2026-03-03.

[18] Intel Corporation. Cyclone V Device Handbook. https://www.intel.com/content/www/us/en/docs/programmable/683375/current/cyclone-v-device-overview.html, 2024. Accessed: 2026-03-03.

[19] Intel Corporation. Cyclone V Device: Embedded Memory User Guide. https://www.intel.com/content/www/us/en/docs/programmable/683179/current/, 2022. Accessed: 2026-03-03.

[20] Intel Corporation. Cyclone V Device: DSP Blocks User Guide. https://www.intel.com/content/www/us/en/docs/programmable/683364/current/, 2022. Accessed: 2026-03-03.

[21] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, 105(12):2295–2329, 2017. doi: 10.1109/JPROC.2017.2761740.

Appendix A

SpMM Background

This appendix provides supporting background on the compressed sparse row (CSR) format and its role in SpMM, for readers less familiar with sparse linear algebra.

CSR Format Details

A sparse matrix A of size $M \times K$ with NNZ nonzeros is stored in CSR as three arrays:

- `rowptr[0..M]`: an array of $M+1$ integers, where `rowptr[i]` is the index into `colidx` and `values` of the first nonzero in row i. By convention, `rowptr[M] = NNZ`.
- `colidx[0..NNZ-1]`: the column index of each nonzero.
- `values[0..NNZ-1]`: the value of each nonzero.

The nonzeros of row i are located at indices `rowptr[i]` through `rowptr[i+1] - 1` in both `colidx` and `values`. The number of nonzeros in row i is `rowptr[i+1] - rowptr[i]`.

Symmetric Quantisation Detail

For a vector $\mathbf{v}$ with maximum absolute value $v_{\max}$, symmetric INT8 quantisation maps each element v_i to a signed 8-bit integer:

$$q_i = \text{clip}\left(\text{round}\left(\frac{v_i \cdot 127}{v_{\max}}\right), -127, 127\right).$$

The scale factor $s = v_{\max}/127$ allows reconstruction: $\hat{v}_i = q_i \cdot s$. The maximum absolute quantisation error is $s/2 = v_{\max}/254$.

Symmetric quantisation does not require a zero-point offset (the zero value maps to zero exactly), which simplifies the accumulation logic in hardware.

Appendix B

Gantt Charts and Project Plan

This appendix records both the planned and the actual project schedule across the two semesters. The planned charts (Figures B.1 and B.2) were produced during the initial planning stage at the start of Semester 1 and represent the intended allocation of effort across literature review, platform setup, baseline RTL, optimisation stages, FPGA deployment, and report writing. The actual charts (Figures B.3 and B.4) record the effort distribution as it unfolded in practice.

The most visible divergences between planned and actual lie in the Semester 2 integration phase. FPGA deployment and timing closure consumed more calendar time than originally allocated, largely as a consequence of the stale `firmware.mif` debugging episode and the routing-congestion issue described in Chapter 8. Report writing also started later than planned, with the bulk of the writing compressed into the final few weeks once the INT8 measurements had been collected and verified. Conversely, unit testbench development in Semester 1 proceeded faster than budgeted, which freed margin that was subsequently absorbed by the integration overrun.

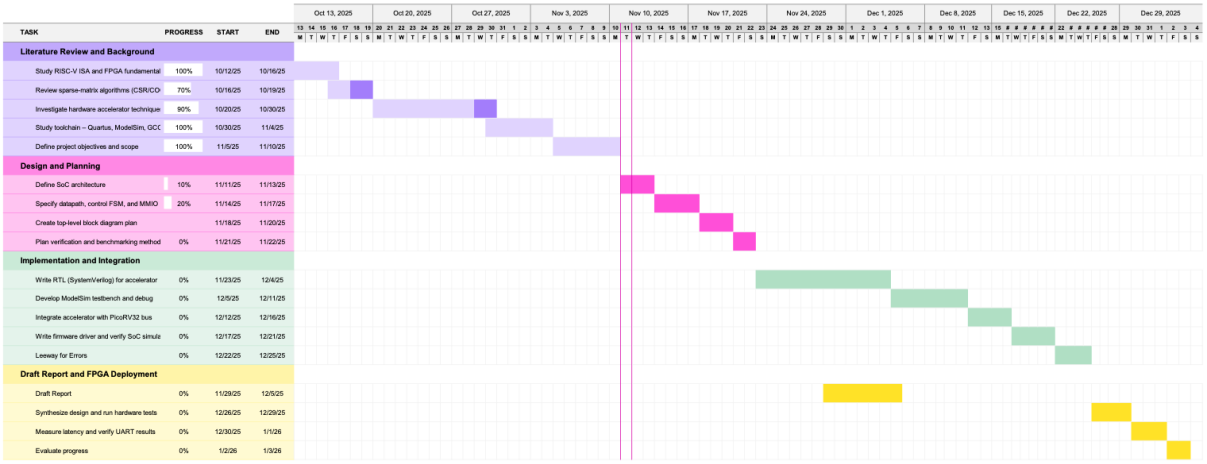

FIGURE B.1: Planned project schedule for Semester 1, covering literature review, platform setup, baseline RTL design, and simulation verification.

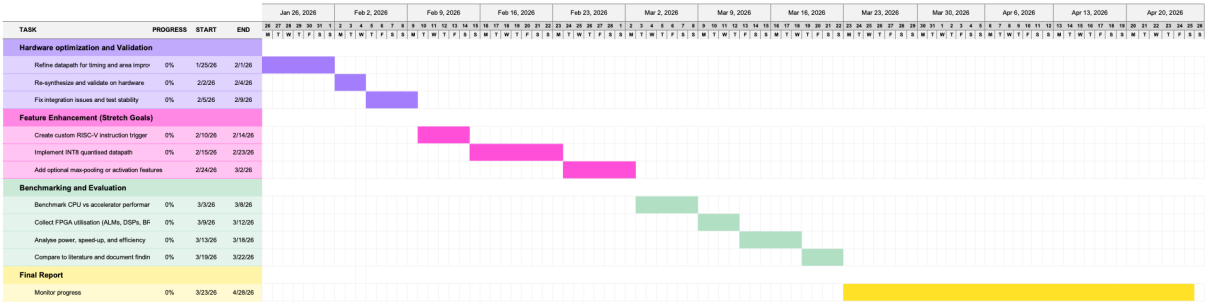

FIGURE B.2: Planned project schedule for Semester 2, covering optimisation stages, firmware benchmark harness, FPGA deployment, and report writing.

Placeholder: actual Semester 1 Gantt chart. Replace with `Actual_Semester1_gantt.png` once available.

FIGURE B.3: Actual effort distribution for Semester 1. Compared to the planned schedule, unit testbench development completed ahead of budget, creating margin that was later absorbed into the Semester 2 integration phase.

Placeholder: actual Semester 2 Gantt chart. Replace with `Actual_Semester2_gantt.png` once available.

FIGURE B.4: Actual effort distribution for Semester 2. FPGA integration and timing closure extended beyond their planned windows due to the stale-artefact and routing-congestion incidents recorded in Chapter 8; report writing was correspondingly compressed into the final weeks.

Appendix C

Benchmark Notes and Stage Relationship

This appendix records the relationship between the evaluation stages discussed in the main report:

- the **baseline accelerator** and **parallel DSP MAC** measurements are historical stage results collected using firmware builds before INT8 packing became the default mainline path;
- the **INT8** stage is the current mainline hardware/software path in the repository; and
- the **Cortex-M0** comparison reuses the same benchmark definitions and the same INT8 accelerator reference cycles, making it a second software baseline, not a different hardware experiment.

The benchmark IDs T1–T12, their matrix dimensions, sparsity levels, and pattern definitions are fixed across all stages. All CPU and accelerator cycle counts reported in Chapter 7 were collected using the PicoRV32 `rdcycle` counter on a physical DE1-SoC board programmed with the synthesised SoC bitstream.

Appendix D

Design and Data Archive

The complete project repository is organised as follows:

- `rtl/`: accelerator RTL, comprising the top-level wrapper (`accel_top.sv`), the control finite state machine (`accel_ctrl.sv`), and the parallel MAC datapath (`accel_datapath.sv`).
- `fpga/rtl/`: synthesisable SoC top-level (`soc_top.sv`) instantiating PicoRV32, the dual-port BRAM, the accelerator, the UART peripheral, and the GPIO interface.
- `fpga/quartus/`: Quartus project files (`de1_soc_spmm.qpf`), pin assignment constraints (`de1_soc_spmm.qsf`), timing constraints (`de1_soc_spmm.sdc`), and the byte-lane memory initialisation files (`firmware[0-3].mif`) generated from the compiled firmware image.
- `sw/driver/`: bare-metal firmware, including the RISC-V assembly startup (`start.S`), benchmark harness (`main.c`), accelerator driver (`spmm_accel.c/.h`), libc stubs (`libc_stubs.c`), linker script (`linker.ld`), `Makefile`, and the Python helper scripts (`bin2hex32.py`, `hex2mif.py`) that convert the compiled ELF into the byte-lane MIF files consumed by Quartus.
- `tb/common/`: shared testbench infrastructure, comprising the clock/reset generator (`clk_reset.sv`), the MMIO SystemVerilog interface (`mmio_if.sv`), the shared utility package (`tb_pkg.sv`), the assertion macros (`tb_macros.svh`), and the shared MMIO register-map header (`tb_regmap.svh`).
- `tb/unit/`: unit testbenches for the individual accelerator modules (`accel_ctrl_tb.sv`, `accel_datapath_tb.sv`, `accel_top_tb.sv`).

- `tb/integ/`: integration testbench (`soc_tb.sv`) that exercises the full synthesisable SoC.
- `sim/`: simulator command scripts (`compile_soc.do`) for invoking the testbenches under ModelSim/Questa.
- `ip/`: third-party IP cores used unmodified, namely the PicoRV32 RV32IM soft core (`ip/picorv32/`) and the Cortex-M0 reference source (`ip/arm_m0/`) used to derive the Cortex-M0 cycle estimates reported in Table 7.5.
- `docs/my_report/`: report LaTeX source, bibliography (`bib/ECS.bib`), figure assets, and chapter files under `chapters/`.
- `README.md`: top-level repository overview, build instructions, and quick-start guide for reproducing the benchmark results.

All source, testbench, firmware, and Quartus project files required to reproduce the measurements reported in Chapter 7 are contained within this repository.